

Digital Contracting Service

Deployment-Leitfaden

Stand 2026-07-31

DCS Deployment-Leitfaden

Dieser Leitfaden nimmt eine DCS-Instanz in Betrieb und hält sie in Betrieb. Er trägt die Administrator-Dokumentationspunkte 1 (Installation und Deployment), 2 (Konfiguration und RBAC) und 5 (Upgrade und Migration) aus SRS §2.6.

Zwei Nachbardokumente ergänzen ihn und werden hier nicht wiederholt:

- **Sicherung und Wiederherstellung:** `docs/backup-integration-guide.md`. Sicherungsinventar, Verfahren und Wiederherstellungsreihenfolge stehen dort, einschließlich der Wechselwirkung zwischen Aufbewahrung und Schlüsselvernichtung.
- **Architektur und Betriebsverhalten:** das technische Handbuch unter `docs/technical-manual/`. Warum eine Komponente existiert, wie ein Ablauf durch das System läuft und was ein Signal bedeutet, steht dort. Dieser Leitfaden sagt, was zu tun ist.

Konkrete Werte, Befehle und Konfigurationsreferenzen stehen ausschließlich hier. Wo ein anderes Dokument einen Wert nennt, gilt dieser hier.

Der Leitfaden hat zwei Teile. **Teil A** ist die Vertragsleistung: der Betrieb auf Kubernetes. **Teil B** beschreibt das lokale Entwickler- und Testsetup und ist für den Betrieb nicht relevant.

Teil A: Betrieb

A1 Zielumgebung und Voraussetzungen

A1.1 Auslieferungsform

DCS wird ausschließlich als Linux-Container-Image für Kubernetes ausgeliefert (DCS-OE-02). Die Deployment-Konfiguration ist ein Helm-Chart (DCS-OE-03), die Auslieferung in einen Cluster erfolgt über ArgoCD (Abschnitt A9), Bau und Veröffentlichung über GitHub Actions (Abschnitt A10).

Zwei Images bilden die Anwendung:

Image	Dockerfile	Inhalt
DCS-Backend	<code>deployment/docker/Dockerfile</code>	Backend-Binary, gebautes Frontend, <code>gendid</code> , <code>hmsign</code> , <code>pacmonitor</code> , SoftHSM2/OpenSC/OpenSSL, die Provisionierungsskripte
pdf-core	<code>pdf-core/Dockerfile</code>	Deterministischer PDF/A-3a -Compiler und -Verifier samt C2PA-Provenance. Die erzeugten Dokumente deklarieren die Konformitätsstufe A der PDF/A-3-Familie (ISO 19005-3), tragen also einen getaggten Strukturbaum und die eingebettete maschinenlesbare Vertragsrepräsentation im selben Dokument

Alle übrigen Komponenten (PostgreSQL, Hydra, NATS, ORCE, DSS, IPFS, Fuseki, Keycloak, Federated Catalogue, Status List Service) kommen aus den Subcharts des Charts mit dort gepinnten Fremd-Images.

Zwei Eigenschaften dieser Auslieferung sind für die Beschaffungs- und Betriebsentscheidung wesentlich:

- **DSS ist ein Community-Image.** Für die DSS-Demo-Webanwendung existiert kein von der EU veröffentlichtes Container-Image; das Subchart zieht deshalb ein Community-Image, gepinnt auf eine feste Version, niemals `latest`. Wer die Herkunft seiner Images selbst verantworten muss, hat zwei Möglichkeiten: das Image aus der EU-Quelle nachbauen und `dss.image.repository/.tag` überschreiben, oder einen eigenen Validierungsdienst betreiben und die Instanz über `dss.url` darauf richten (A5.6).
- **Fuseki und der Federated Catalogue sind mitgelieferte Bequemlichkeit.** Sie hängen als Abhängigkeit im Chart, damit eine Instanz „mit Batterien“ startet und ohne fremde Vorleistung vollständig arbeitet. Für den Betrieb sind sie keine Vorgabe. Betreibt eine Organisation bereits einen Federated Catalogue, bleibt `fcservice.enabled` aus und die Instanz wird über `federatedCatalogue.*` auf den vorhandenen gerichtet (A5.5).

A1.2 Cluster-Voraussetzungen

Voraussetzung	Warum
Kubernetes-Cluster mit Ingress-Controller	Die <code>.well-known</code> -Dokumente, die UI und die API müssen extern erreichbar sein: Peers lösen die Instanz über ihre Föderationsdokumente auf, Wallets rufen die API während der Signaturzeremonie direkt an, und Zielsysteme melden über die API zurück. Die mitgelieferten Beispielwerte für Issuer und <code>sharedServices</code> verwenden <code>nginx.ingress.kubernetes.io/*</code> -Annotationen und setzen damit ingress-nginx voraus; unter einem anderen Controller sind die Rewrite-Regeln zu ersetzen.
Ziel-Namespace existiert bereits	<code>deploy.sh</code> legt ihn nicht an. Das ist Absicht: das Skript arbeitet mit Credentials, die keine clusterweiten Objekte sehen.
StorageClass für PVCs	Postgres, IPFS, der HSM-Token, ORCE, Fuseki und fc-postgres beanspruchen je einen Claim. Für die Verschlüsselung im Ruhezustand (DCS-NFR-SEC-14) ist eine verschlüsselte Klasse pro Umgebung zu setzen, siehe A5.4.
RBAC: <code>roles / rolebindings</code> im Namespace anlegen dürfen	Der HSM-Provisionierungs-Hook und die ACME-Erneuerung bringen eigene ServiceAccount/Role/RoleBinding mit. Ist das nicht erlaubt, gibt es einen Betriebsmodus ohne API-Zugriff (A6.5).
Ressourcenrahmen	Eine vollständige Instanz sind rund zehn Workloads und zehn Services. Unter einer <code>ResourceQuota</code> ist besonders <code>limits.cpu</code> zu prüfen; unter einer <code>LimitRange</code> der Minimalwert pro Container. Beides erzeugt Fehlerbilder, die nicht wie Fehler aussehen (A13).
TLS-Zertifikat für den öffentlichen Host	Ein Peer löst diese Instanz über <code>https://<host>/.well-known/did.json</code> auf. cert-manager ist der Normalfall; für Cluster, in denen cert-manager den HTTP-01-Solver nicht schedulen kann, liefert das Chart einen CronJob-Fallback (<code>acmeRenewal</code> , siehe A5.6).

A1.3 Werkzeuge auf dem Arbeitsplatz

`helm` und `kubectl` müssen im `PATH` liegen, sonst bricht `deploy.sh` ab. `curl` wird für die externen Prüfungen benötigt. `docker` nur, wenn Images lokal gebaut werden.

Chart-Version und Anwendungsversion stehen in `deployment/helm/Chart.yaml` (derzeit `version: 0.2.2`, `appVersion: "1.16.0"`). `appVersion` ist der Fallback für `image.tag` und `pdfCore.image.tag`, wenn dort nichts gesetzt ist.

A2 Images beziehen und bauen

A2.1 Bezug aus der Harbor-Registry

Der Release-Workflow `.github/workflows/publish.yml` baut beide Images und veröffentlicht das Chart. Die Registry-Adresse ist das GitHub-Actions-Secret `HARBOR_HOST`, die Zugangsdaten sind `HARBOR_CREDENTIALS` (Base64 eines JSON-Objekts `{username: password}`, kein Klartextpasswort). Projekt und Benutzer stehen in `deployment/harbor.config`:

```
PROJECT=dcs
USER=dcs
```

Harbor erwartet die Robot-Account-Form `robot$<user>`, nicht den nackten `USER`-Wert aus dieser Datei; die Ableitung übernimmt `harborconfig.sh` aus `eclipse-xfsc/dev-ops`.

Das Chart wird als OCI-Artefakt nach `oci://<HARBOR_HOST>/dcs/charts` gepusht. Ein Betreiber bezieht es damit über:

```
helm pull oci://<HARBOR_HOST>/dcs/charts/digital-contracting-service --version <chart-version>
```

Für den Pull der Images aus einer privaten Registry gibt es zwei Wege: einen vorhandenen Pull-Secret über `imagePullSecrets` referenzieren, oder das Chart eines anlegen lassen (`dockerRegistry.create=true` mit `server`, `username`, `password`; der Secret-Name ist `dockerRegistry.secretName`, Vorgabe `docker-registry`). Die zweite Variante schreibt die Zugangsdaten in ein Werte-Set. Sie gehört deshalb in einen Secret-Manager-Aufruf über `--set` und nicht in eine Werte-Datei.

A2.2 Lokaler Bau

Voraussetzungen: Docker, Zugriff auf das Repository-Wurzelverzeichnis (beide Dockerfiles brauchen es als Build-Kontext).

Schritte:

1. DCS-Backend-Image bauen:

```
DOCKER_REGISTRY=<registry> DOCKER_REPO=<repo> bash deployment/docker/build-image.sh <tag>
```

Ohne `DOCKER_REGISTRY` und `DOCKER_REPO` baut das Skript `digital-contracting-service:<tag>` lokal und pusht nicht. Mit beiden Variablen baut es `<registry>/<repo>/digital-contracting-service:<tag>`, taggt zusätzlich `:latest` und pusht beide.

2. pdf-core-Image bauen:

```
docker build -f pdf-core/Dockerfile -t <registry>/<repo>/pdf-core:<tag> .
```

Verifikation: Das Backend-Image enthält alle vier Binaries und die Provisionierungsskripte. Beobachtbares Signal:

```
docker run --rm --entrypoint ls <image> -1 /app/dcs /app/gendid /app/hsm-sign /app/pacmonitor /app/scripts/hsm-provision.sh
/app/scripts/c2pa-cert-provision.sh
```

Alle sechs Pfade müssen ausgegeben werden. Fehlt `/app/gendid`, kann der HSM-Provisionierungs-Hook kein `did.json` erzeugen und die Instanz startet nie durch; fehlt `/app/pacmonitor`, schlägt der Compliance-CronJob bei jedem Lauf fehl.

Fehlerbilder:

Symptom	Ursache	Behebung
go: module lookup disabled by GOPROXY=off im Build	Der Codegenerator wird im Build hermetisch aus dem Modul-Cache gestartet, seine eigene Abhängigkeit fehlt aber in <code>backend/go.sum</code>	Am Repository beheben (siehe unten), nicht am Build
Image startet, <code>dlopen</code> von <code>libsoftasm2.so</code> schlägt fehl	Ein Build mit <code>CGO_ENABLED=0</code> . Die PKCS#11-Schicht ist cgo-only.	Nur das mitgelieferte Dockerfile verwenden; es baut mit <code>CGO_ENABLED=1</code> auf einer glibc-Toolchain.

Zum ersten Fall: das ist ein Repository-Zustand, der zu beheben ist. Der Codegenerator wird im Build als Werkzeug ausgeführt, ist in `backend/go.mod` aber nicht als Werkzeugabhängigkeit deklariert. Pre-Commit-Hook und Lint-Workflow erzwingen `go mod tidy`, und `go mod tidy` hat damit keinen Grund, die Abhängigkeiten des Generators in `go.sum` zu halten. Der hermetische Generierungsschritt verliert sie dann. Die Behebung ist einmalig und gehört ins Repository:

1. Im Verzeichnis `backend/` den Generator als Werkzeugabhängigkeit eintragen:

```
go get -tool goa.design/goa/v3/cmd/goa
```

2. Die geänderten `backend/go.mod` und `backend/go.sum` committen.

Verifikation: `backend/go.mod` enthält danach einen `tool`-Block, der den Generator nennt, und ein anschließendes `go mod tidy` verändert `go.sum` nicht mehr. Der Build läuft ohne manuelles Nachziehen durch.

Bis dahin gilt: Ein Image-Bau, der pro Lauf eine Handreichung braucht, ist nicht reproduzierbar. Der Betreiber soll ihn nicht von Hand reparieren.

A2.3 Tag-Disziplin

`image.tag` und `pdfCore.image.tag` sind in Produktion auf eine explizite Release-Version zu pinnen, nie leer und nie `latest`. Zwei Gründe, beide real eingetreten:

- Ein leerer Tag fällt auf `Chart.AppVersion` zurück; das ist eine Chart-Eigenschaft, keine Deployment-Entscheidung.
- Bei aktivierter HSM-Provisionierung nutzt der Hook-Job standardmäßig dasselbe Image wie die Anwendung. Läuft die Anwendung mit `pullPolicy: Always`, der Job aber mit `IfNotPresent` auf einem beweglichen Tag, kann der Node-Cache einen älteren Schlüsselsatz provisionieren als die Anwendung erwartet. Die Anwendung bricht dann beim Start auf einem Schlüssel ab, den der Provisionierer nie angelegt hat. Deshalb ist `pkcs11.provisioning.image.pullPolicy` per Vorgabe leer und folgt `image.pullPolicy`.

A3 Chart-Aufbau und Subchart-Abhängigkeiten

Das Elternchart heißt `digital-contracting-service` und liegt unter `deployment/helm`. Es rendert das Backend-Deployment, das pdf-core-Deployment, Services, Ingress, die Provisionierungs-Hooks und die CronJobs. Alles Weitere kommt aus Subcharts.

Subchart	Alias	Aktiviert durch	Rolle
postgresql	kein Alias	postgresql.enabled	Mitgelieferter Datenbankserver
hydra	kein Alias	hydra.enabled	OAuth2/OIDC für Browser- und Maschinen-Login
nats	kein Alias	nats.enabled	Event-Bus
orce	kein Alias	orce.enabled	Node-RED-Flows: Trust-PDP, Archiv-Notar, TSA, Contract-Target, Audit-Executor
dss	kein Alias	dss.enabled	EU-DSS-Signaturvalidierung
federated-catalogue	fcservice	fcservice.enabled	Federated-Catalogue-Dienst
keycloak	fcKeycloakApp	fcservice.enabled	Identitätsanbieter nur für den Federated Catalogue
fuseki	fcFuseki	fcservice.enabled	Graph-Store des Federated Catalogue
fc-postgres	fcPostgres	fcservice.enabled	Datenbank für fc-service und dessen Keycloak
ipfs	ipfs	ipfs.enabled	Kubo-Knoten, hält alle Artefakt-Chifftrate
ipfs-document-manager	ipfsDocumentManager	ipfsDocumentManager.enabled	Tenant-Gateway vor IPFS
statuslist-service	statuslistService	statuslistService.enabled	XFSC-Status-Listen-Dienst. Per ADR-34 zur Entfernung bestimmt, derzeit noch Startvoraussetzung des Backends (A5.6)
kube-prometheus-stack	kein Alias	monitoring.enabled	Prometheus/Grafana/Alertmanager
traefik	kein Alias	traefik.enabled	Ingress-Controller, nur für Testcluster

Die vier `fc*`-Subcharts hängen gemeinsam an `fcservice.enabled`. Eine Instanz, die einen **entfernten** Federated Catalogue nutzt, setzt stattdessen nur `federatedCatalogue.enabled`. Das ist der Konfigurationsblock des DCS-Backends für ausgehende Aufrufe, unabhängig davon, wo der Katalog läuft.

A3.1 Abhängigkeiten bauen

```
helm dependency build --skip-refresh deployment/helm
```

Das ist bei jedem Deployment auszuführen, nicht nur beim ersten. Helm bevorzugt ein gepacktes `charts/*.tgz` gegenüber dem Quellverzeichnis; ein Archiv aus einem früheren Bau überdeckt sonst Änderungen an `charts/<name>/` und bringt alte Templates aus. `deploy.sh` tut das von sich aus.

`kube-prometheus-stack` und `traefik` kommen aus externen Repositories. Wenn sie noch nicht bekannt sind:

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo add traefik https://traefik.github.io/charts
```

`--skip-refresh` baut aus dem committeten `Chart.lock`. Ohne die beiden Repositories schlägt der Bau bei den externen Abhängigkeiten fehl.

Wird `monitoring.enabled` gesetzt, sind die CRDs des Prometheus-Stacks vorab anzulegen. Sie sind nicht Teil des Release:

```
helm show crds deployment/helm/charts/kube-prometheus-stack-*.tgz | kubectl apply --server-side -f -
```

A4 Umgebungsmatrix der values-Dateien

Die Chart-eigene `values.yaml` wird immer zuerst angewandt. Jede Datei unten liegt darüber. Reihenfolge zählt: spätere `--values` überschreiben frühere.

Datei	Umgebung	Setzt voraus	Muss beim Zielcluster überschrieben werden
values.yaml	Basis, immer aktiv	nichts	Nichts direkt; sie ist bewusst nicht deploybar (keine Registry, kein Host, keine Identität)
values.prod.yaml	Produktion (Skelett)	Externer PostgreSQL, echtes HSM, vorab angelegte Secrets	image.repository / tag, alle Hostnamen, pkcs11.modulePath / tokenLabel, federatedCatalogue.apiUrl, fcKeycloak.realmURL, systemToken, identity / DID-Quelle, oid4vp.trust, Storage-Klassen. Siehe A4.1
values.acceptance.yaml	Abnahme/Staging (Skelett)	Vorab angelegte Secrets; SoftHSM2 im Cluster	Dasselbe wie Produktion, außer PKCS#11 (bleibt SoftHSM2). Siehe A4.1
values.dev.yaml	Lokale Entwicklung, Instanz A	Rancher Desktop o. ä. mit NodePort-Weiterleitung auf localhost	Nicht für Cluster-Betrieb geeignet: replicaCount: 0, Backend und pdf-core laufen nativ
values.dev2.yaml	Lokale Entwicklung, Instanz B	wie oben, zweiter Portbereich	wie oben
values.bdd.yaml	kind-Testcluster, Instanz A	kind mit hostPort 18080, lokal geladene Images	Nicht für Betrieb: image.pullPolicy: Never, DCS_ALLOW_DEV_TRUST=true, Trafik mit hostPort
values.bdd2.yaml	kind-Testcluster, Instanz B	Instanz A läuft bereits im selben Namespace	wie oben
values.issuer.yaml	DEMO , Credential-Issuer neben Instanz A	Eigenes Helm-Release des orce-Subcharts	ingress.hosts[].host, ingress.className, ingress.tls
values.issuer2.yaml	DEMO , Credential-Issuer neben Instanz B	wie oben	wie oben
values.pid-issuer.yaml	DEMO , PID-Issuer, einmal je Föderation	Eigenes Helm-Release des orce-Subcharts	ingress.hosts[].host, ingress.className, ingress.tls

Die drei Issuer-Dateien tragen die Kennzeichnung **DEMO**, weil das, was sie starten, ein Demonstrations-Issuer ist. Er prägt Credentials, ohne jemanden zu prüfen. Das gilt für die Vollmacht ebenso wie für den Personenidentitätsnachweis. Solche Issuer gehören in eine Vorführung, eine Abnahme oder einen Testaufbau. An einen Vertrag mit Rechtswirkung gehören sie nicht. Was ein produktiver Betrieb stattdessen braucht, steht in A7.

A4.1 values.prod.yaml und values.acceptance.yaml sind Skelette

Beide Dateien beschreiben das **Muster** einer Umgebung. Ein einsatzfähiges Deployment sind sie nicht. Sie tragen Platzhalter (dcs.example.org, [registry], leere apiUrl) und lassen mehrere zwingende Werte offen. Ein helm upgrade --install mit einer dieser Dateien ohne weitere Overlays führt in dieser Reihenfolge zu:

1. **Render bricht ab.** systemToken ist in keiner der beiden Dateien deklariert, das Backend-Template fordert ihn per required an. Die Fehlermeldung nennt den Grund.
2. **Nach Ergänzung: Render bricht erneut ab.** oid4vp.trust.enabled ist in beiden Dateien nicht gesetzt und damit false; damit rendert die Instanz ohne Vertrauensdokument, und jede Credential-Vorlage schlägt zur Laufzeit fehl. Setzt man oid4vp.trust.enabled=true und lässt existingConfigMap leer, verweigert das Chart den Render mit einem Hinweis auf das Dev-Fixture (siehe A7.4).
3. **Nach Ergänzung: Pod stürzt beim Start ab.** identity.enabled ist in beiden Dateien nicht gesetzt. Ohne sie rendert das Deployment kein DCS_DID, das Backend kann sein DID-Dokument nicht lesen und protokolliert Could not read did document.
4. **Katalog-Anmeldung schlägt fehl.** Beide Dateien setzen federatedCatalogue.oauth.clientID: "dcs-fc-client" (ebenso die Chart-Vorgabe). Der Realm, den das mitgelieferte Keycloak importiert, legt diesen Client nicht an; er heißt dort federated-catalogue. Gegen den mitgelieferten Katalog quittiert die Token-Anforderung deshalb mit 401. Wer einen externen Keycloak betreibt, setzt hier den dort tatsächlich angelegten Client.
5. **values.prod.yaml zusätzlich:** pkcs11.modulePath und pkcs11.tokenLabel sind leer und pkcs11.provisioning.enabled=false. Ohne echte Werte findet das Backend kein Token. Ebenso ist pdfCore.signing.existingSecret zwar auf dcs-prod-pdf-core-signing gesetzt, dieses Secret muss aber außerhalb des Charts erzeugt werden. Der Provisionierungs-Hook läuft in diesem Modus nicht.

So sind die Dateien gemeint. Hostnamen, Registry, Identität und Secret-Anbindung sind Deployment-Zustand und gehören in eine Werte-Datei **außerhalb dieses Repositories**. Behandle die beiden Dateien als Checkliste, nicht als Konfiguration.

A4.2 Umgebungsabhängige Werte

Die folgenden Werte stimmen jeweils nur unter einer bestimmten Cluster-Variante und sind beim Wechsel zu prüfen:

Wert	Gilt unter	Wirkung anderswo
<code>traefik.ports.web.hostPort: 18080</code> (<code>values.bdd.yml</code>)	kind mit passender <code>extraPortMappings</code> -Konfiguration	Auf k3s belegt Klippers svclb den Port bereits; der Traefik-Pod bleibt <code>Pending</code>
<code>ingress.className: traefik</code> (<code>values.bdd.yml</code>)	Testcluster mit dem mitgelieferten Traefik	In Produktion ist die Klasse des vorhandenen Controllers zu setzen
<code>nginx.ingress.kubernetes.io/*</code> -Annotationen (Issuer-Werte, <code>sharedServices</code>)	ingress-nginx	Unter Traefik oder Istio wirkungslos; die Pfad-Rewrites müssen ersetzt werden
Reihenfolge <code>/.well-known/did.json</code> vs. Hydras <code>/.well-known</code>	ingress-nginx bevorzugt den exakteren Pfad	Unter Controllern ohne diese Präferenz sind die drei Dokumente wie in <code>values.bdd.yml</code> als vollständige Pfade zu listen
<code>image.pullPolicy: Never</code> (BDD-Werte)	kind mit <code>kind load docker-image</code>	In jedem anderen Cluster startet kein Pod
leere <code>storageClassName</code>	Cluster mit sinnvoller Default-Klasse	Ohne Default bleibt der Claim ungebunden und der Pod <code>Pending</code>
<code>pkcs11.provisioning.persistence.size: 1Gi</code>	CSI-Treiber mit Mindestgröße	Der Token braucht rund 10 MiB; 1 Gi steht dort, weil mehrere Treiber darunter nicht provisionieren
<code>service.type: NodePort</code> mit festen Ports (Dev-Werte)	Rancher Desktop	In einem geteilten Cluster kollidieren die Ports

A5 Konfigurationsreferenz nach Wirkung

Aufgeführt sind nur Werte, für die im Chart ein rendernder **und** im Code ein lesender Ort belegt ist. Ein Wert, der sich zwar setzen lässt, aber von nichts gelesen wird, steht hier nicht. Er würde einen Knopf vortäuschen, der nichts bewirkt.

A5.1 Identität, Routing und öffentliche Adresse

Wert	Gerenderte Wirkung	Wirkung im System
<code>route.basePath</code>	<code>DCS_API_PATH = <basePath>/api</code> , <code>DCS_UI_PATH = <basePath>/ui</code> , Ingress-Pfad	Unter welchem Präfix API und UI liegen. Peers und Callbacks bilden ihre URLs daraus.
<code>paths.api</code> , <code>paths.ui</code>	Überschreiben die beiden abgeleiteten Pfade einzeln	Nur nötig, wenn API und UI nicht unter demselben Präfix liegen
<code>route.publicBaseURL</code>	<code>DCS_PUBLIC_BASE_URL</code> (wörtlich) und <code>DCS_PUBLIC_URL</code> (Schema + <code>didHostname</code> + Pfad)	Beide sind beim Start zwingend. <code>DCS_PUBLIC_URL</code> ist die Basis jeder absoluten IRI in erzeugten Dokumenten; <code>DCS_PUBLIC_BASE_URL</code> die öffentliche API-Basis für Wallet-Rückläufe.
<code>route.didHostname</code>	Host-Anteil des <code>did:web</code> -Bezeichners und der Anker-URL	Nur zu setzen, wenn der <code>did:web</code> -Hostname nicht der Host aus <code>publicBaseURL</code> ist. Leer heißt: Host aus <code>publicBaseURL</code> , sonst <code>localhost:<service.port></code> .
<code>signing.issuerDID</code> bzw. <code>signing.issuerDIDSecretRef</code>	<code>ISSUER_DID</code>	Die DID dieser Instanz. Der Provisionierungs-Hook verwendet sie wörtlich für <code>did.json</code> ; nur wenn sie leer ist, leitet er eine aus dem Hostnamen ab. Das ist der Weg, mehrere Instanzen unter einem Hostnamen mit Pfadsegmenten zu betreiben.
<code>identity.enabled</code>	Rendert <code>DCS_DID</code> und mountet die Identitäts-Quelle	Zwingend für jedes In-Cluster-Deployment. Ohne sie kein <code>DCS_DID</code> , ohne <code>DCS_DID</code> kein DID-Dokument, ohne DID-Dokument kein Start.
<code>identity.secretName</code> , <code>identity.mountPath</code>	Name des Secrets bzw. Mount-Pfad	Vorgabe: <code><fullname>-identity</code> unter <code>/app/identity</code> . Der Name wird aus dem Release abgeleitet, damit zwei Releases in einem Namespace nicht kollidieren.
<code>ingress.enabled</code> , <code>.className</code> , <code>.hosts</code> , <code>.tls</code> , <code>.annotations</code> , <code>.pathType</code>	Ingress-Objekt	<code>ingress.enabled=true</code> ohne mindestens einen Eintrag in <code>hosts</code> bricht den Render ab.
<code>service.port</code>	Container-Port, Service-Port, Ingress-Backend-Port	Vorgabe 8991

A5.2 Datenhaltung und Messaging

Wert	Gerenderte Wirkung	Wirkung im System
database.secretRef.name / .key	DATABASE_URL aus einem Secret	Bevorzugter Weg. Gesetzt, gewinnt er gegen alle anderen database.*-Felder.
database.url	DATABASE_URL als Literal	Nur ohne secretRef
database.port/user/password/name/sslmode	Bestandteile der abgeleiteten DSN	Greifen nur, wenn weder secretRef noch url gesetzt sind und ein Postgres-Host auflösbar ist
postgresql.enabled	Mitgelieferter Server; Host wird <release>-postgresql	Ohne ihn muss serviceDiscovery.postgresqlHost oder eine vollständige DSN gesetzt sein
postgresql.additionalDatabases	Weitere Datenbanken beim ersten Start	Jede Komponente mit eigener DSN auf diesem Server braucht hier einen Eintrag. Hydra führt nur hydra migrate sql aus und legt keine Datenbank an. Ohne Eintrag bleibt Hydra dauerhaft nicht bereit.
postgresql.persistence.enabled	PVC statt emptyDir	Vorgabe ist emptyDir. Damit verliert ein Pod-Neustart jeden Vertrag. Für alles jenseits eines Wegwerf-Clusters einzuschalten.
messaging.secretRef / messaging.natsURL / messaging.natsPort	NATS_URL	Ohne nats.enabled und ohne serviceDiscovery.natsHost bleibt die URL leer
ipfs.enabled, ipfs.persistence.*	Kubo-Knoten und dessen Volume	Das Volume hält alle gepinnten Artefakt-Chifftrate. Ein Verlust ist ein Audit-Trail-Verlust, nicht nur ein Dokumentverlust.
ipfs.tenantBaseURLSecretRef	IPFS_TENANT_BASE_URL aus einem Secret	Der Weg, die Tenant-Adresse aus einem Secret statt aus der Werte-Datei zu beziehen
ipfsClient.tenantBaseURL, ipfsClient.tenantName	IPFS_TENANT_BASE_URL explizit bzw. Tenant-Pfad	Ohne Angaben abgeleitet als http://<release>-ipfs-document-manager:8080/v1/tenants/<tenantName>
ipfsClient.mfsBaseURL	IPFS_MFS_BASE_URL	Ohne Angabe abgeleitet als http://<release>-ipfs:5001

Beide IPFS-Variablen sind beim Start zwingend; fehlen sie, bricht das Backend mit `IPFS configuration missing` ab. Die `ipfs.*`-Werte konfigurieren den **mitgelieferten** Kubo-Knoten. Wer stattdessen einen vorhandenen IPFS-Knoten anbindet, setzt die Adressen über `ipfsClient.tenantBaseURL` und `ipfsClient.mfsBaseURL`. Aus diesen beiden Feldern entstehen die Umgebungsvariablen.

A5.3 Authentifizierung, Rollen und Maschinen-Identitäten

Wert	Gerenderte Wirkung	Wirkung im System
hydra.enabled	Hydra-Subchart	Ohne Hydra kein Browser-Login und keine Maschinen-Token
hydra.config.selfIssuerURL	HYDRA_PUBLIC_ISSUER_URL	Die Adresse, die Hydra in jede Redirect-URL stempelt. Muss extern erreichbar sein und zum Ingress-Host passen.
hydra.config.internalIssuerURL	HYDRA_INTERNAL_ISSUER_URL	Adresse, über die das Backend Hydra im Cluster erreicht. Leer heißt <code>http://<release>-hydra:<publicPort></code> .
hydra.config.loginURL, .consentURL, .logoutURL	Hydras Weiterleitungsziele	Müssen auf die DCS-UI bzw. deren Callback-Pfade zeigen
hydra.config.dev	Hydra-Dev-Modus	Nur zulässig, wenn selfIssuerURL <code>http://</code> ist. In Produktion <code>false</code> .
hydra.clients[0]	HYDRA_CLIENT_ID, HYDRA_CLIENT_SECRET, HYDRA_REDIRECT_URI, HYDRA_POST_LOGOUT_REDIRECT_URI	Nur der erste Eintrag wird in das Backend gerendert. Er ist der Browser-Client. Weitere Einträge existieren in Hydra, aber das Backend kennt sie nicht als eigenen Login-Client.
hydra.route.paths	Ingress-Regeln nach Hydra	Vorgabe <code>/oauth2</code> und <code>/.well-known</code> . Der zweite Eintrag ist der Grund, warum die drei DCS- <code>/.well-known</code> -Dokumente separat und exakt geroutet werden.
hydra.secrets.system	Hydras Systemsecret	Es gibt in dieser Chart-Version keine secretKeyRef - Entsprechung. Wert über <code>--set</code> aus dem Secret-Manager injizieren, nie in eine Werte-Datei schreiben.
systemClients	DCS_SYSTEM_CLIENTS (JSON-Array)	Saat für die Maschinen-Identitäten. Jeder Eintrag braucht <code>client_id</code> , <code>participant_id</code> und <code>roles</code> . Die Rollen kommen aus der Konfiguration, nie aus dem Token; ein Client kann nicht mehr anfordern, als ihm gegeben ist. Ein unbekannter Rollenname bricht den Start ab.
systemToken	DCS_SYSTEM_TOKEN	In-Cluster-Dienstcredential der Hintergrund-PDF-Regeneration. Siehe A6.6.

Gültige Rollennamen (exakte Schreibweise; ein anderer Wert stoppt den Start):

Menschliche Rollen: Template Creator, Template Reviewer, Template Approver, Template Manager, Contract Creator, Contract Reviewer, Contract Approver, Contract Manager, Contract Negotiator, Contract Signer, Contract Observer, Archive Manager, Auditor, Sys. Administrator, Compliance Officer, Integration Manager, Process Orchestrator, Validator.

Systemrollen: Sys. Contract Creator, Sys. Contract Reviewer, Sys. Contract Approver, Sys. Contract Manager, Sys. Contract Signer, Sys. Auditor, Contract Target System.

Drei Eigenschaften dieser Liste sind betrieblich relevant:

- **Sys. Contract Signer trägt keine Signaturberechtigung.** Die Klasse existiert, damit die Verweigerung nachweisbar ist. Eine Signatur braucht immer eine natürliche Person mit Wallet.
- **Systemrollen allein reichen für den Katalog nicht.** Die drei Katalog-Endpunkte (/catalogue/template/retrieve, /catalogue/template/retrieve/{did}, /catalogue/template/search) verlangen eine der Rollen Template Manager, Contract Creator, Contract Reviewer, Contract Approver, Contract Manager oder Contract Signer. Alle sechs sind menschliche Rollen. Ein Maschinen-Token, das nur Systemrollen trägt, erhält dort insufficient permissions. Die Prüfung ist ein reiner Namensabgleich über die Rollenliste und unterscheidet keine Rollenklassen: Wird einem Maschinen-Client eine der genannten menschlichen Rollen zugewiesen, über systemClients oder über die Maschinen-Identitäts-Verwaltung, dann wird er am Katalog zugelassen. Weder die Startprüfung noch die Verwaltung verweigert das. Wer einem Maschinen-Client Katalogzugriff geben will, tut das also bewusst und trägt die Folge: Ein Katalogeintrag, den eine Maschine gezogen hat, ist keinem Menschen zurechenbar.
- **Contract Target System wird nicht über systemClients vergeben,** sondern über die Registrierung eines Zielsystems (siehe contractTargets).

Ein Eintrag in systemClients allein reicht nicht: derselbe client_id muss auch unter hydra.clients existieren, sonst authentifiziert sich der Client nirgends. Umgekehrt authentifiziert sich ein Client, der nur unter hydra.clients steht, erfolgreich und wird danach überall abgewiesen.

systemClients ist eine **Saat** und keine Laufzeitquelle. Die Einträge werden beim Start in die Maschinen-Identitäts-Registry übernommen, damit eine Instanz ihre Aufrufer hat, bevor sich jemand anmeldet. Danach werden Maschinen-Identitäten in der Verwaltungsoberfläche administriert.

A5.4 Signaturmaterial und PKCS#11

Wert	Gerenderte Wirkung	Wirkung im System
<code>pkcs11.modulePath</code>	PKCS11_MODULE_PATH	Pfad zum PKCS#11-Modul im Container. Vorgabe <code>/usr/lib/softhsm/libsofthsm2.so</code> .
<code>pkcs11.tokenLabel</code>	PKCS11_TOKEN_LABEL	Vorgabe <code>dc</code>
<code>pkcs11.pinSecretRef.name</code> / <code>.key</code>	PKCS11_PIN aus einem Secret	Ist <code>name</code> leer, legt das Chart ein Secret <code><fullname>-hsm-pin</code> aus <code>pkcs11.pin</code> an. Für Produktion immer ein vorab angelegtes Secret referenzieren.
<code>pkcs11.pin</code>	Inhalt des automatisch angelegten Secrets	Wird ignoriert, sobald <code>pinSecretRef.name</code> gesetzt ist
<code>pkcs11.provisioning.enabled</code>	HSM-PVC, Provisionierungs-Hook, SOFTHSM2_CONF, Token-Mount	<code>true</code> co-deployt einen SoftHSM2-Softwaretoken. Das ist Dev/Staging/CI, nie ein Produktions-HSM.
<code>pkcs11.provisioning.tokenDir</code>	Mount-Pfad des Token-Volumes und Wert von SOFTHSM2_CONF	Vorgabe <code>/var/lib/softhsm</code>
<code>pkcs11.provisioning.soPin</code>	SO-PIN bei der Token-Initialisierung	Nur bei der Erstinitialisierung verwendet
<code>pkcs11.provisioning.publishSecrets</code>	Ob der Hook Secrets über die Kubernetes-API schreibt	Siehe A6.5
<code>pkcs11.provisioning.persistence.size/accessModes/storageClassName</code>	Der Token-PVC	<code>storageClassName</code> ist der Ort, an dem die Verschlüsselung des Schlüsselverzeichnisses entschieden wird (DCS-NFR-SEC-14)
<code>pkcs11.provisioning.resources</code>	Ressourcenblock des Hook-Jobs	Auf Namespaces mit <code>limits.cpu</code> -Quota zu setzen
<code>pkcs11.provisioning.image.repository/tag/pullPolicy</code>	Image des Hook-Jobs	Leer heißt: dasselbe Image wie die Anwendung
<code>signing.tsaURL</code>	TSA_URL	Beim Start zwingend. Leer bricht ab. Siehe A5.8 dazu, worauf diese URL per Vorgabe zeigt.
<code>signing.tsaTrustCertSecret</code>	TSA_TRUST_CERT_FILE = <code>/etc/dcs/tsa-trust/tsa-cert.pem</code> , plus Secret-Mount	Ersetzt das im Binary eingebettete TSA-Zertifikat. Das Secret muss den Schlüssel <code>tsa-cert.pem</code> tragen. Ein Wechsel der Zeitstempel-Autorität ist damit am Backend eine Konfigurationsänderung ohne Neubau .
<code>pdfCore.signing.existingSecret</code>	Name des Secrets mit der C2PA-Zertifikatskette	Erforderlich, sobald <code>pkcs11.provisioning.enabled=false</code>
<code>pdfCore.signing.existingSecretX5ChainKey</code>	Schlüsselname in diesem Secret	Vorgabe <code>x5chain-pem</code>

Drei Storage-Klassen tragen Material im Ruhezustand und sind pro Umgebung auf eine verschlüsselte Klasse zu setzen:

`postgresql.persistence.storageClassName` (Verträge und gewappte CEKs), `ipfs.persistence.storageClassName` (alle Artefakt-Chiffre) und `pkcs11.provisioning.persistence.storageClassName` (die privaten SoftHSM2-Schlüssel). `deploy.sh` prüft nach dem Deployment, ob der gebundene Claim tatsächlich diese Klasse trägt. Ein PVC behält die Klasse, mit der er entstanden ist; eine spätere Werte-Änderung bleibt wirkungslos.

A5.5 Föderation und Vertrauen

Dieser Abschnitt hat drei Werte-Gruppen: das Trust-Gate gegenüber Peers, das Vertrauensdokument für vorgelegte Credentials und die Anbindung an den Federated Catalogue. Die Tabellen nennen Wert und gerenderte Wirkung; was ein Wert im Betrieb bedeutet, steht als Fließtext unter der jeweiligen Tabelle.

Trust-Gate gegenüber Peers

Wert	Gerenderte Wirkung
<code>federation.trustPdpURL</code>	DCS_TRUST_PDP_URL, nur wenn gesetzt

Der Trust-Gate ist der Policy-Endpunkt, gegen den jede eingehende und ausgehende föderierte Interaktion geprüft wird. Er ist **fail-closed**: Solange die URL leer ist oder der Endpunkt nicht mit 2xx antwortet, versendet die Instanz nichts und nimmt nichts an. Sie meldet das nicht von sich aus, der Ausfall sieht also aus wie Stille. Die Prüfung dafür steht in A12.2. Jede Instanz konsultiert ausschließlich ihren eigenen Endpunkt; einen

gemeinsamen Gate über eine Föderation hinweg gibt es nicht.

Vertrauensdokument für vorgelegte Credentials

Wert	Gerenderte Wirkung
<code>oid4vp.trust.enabled</code>	Rendert <code>OID4VP_TRUST_DATA_PATH</code>
<code>oid4vp.trust.existingConfigMap</code>	ConfigMap-Name des Vertrauensdokuments
<code>oid4vp.trust.existingConfigMapKey</code>	Schlüssel darin
<code>oid4vp.trust.existingConfigMapMountPath</code>	Mount-Pfad im Container
<code>oid4vp.trust.dataPath</code>	Pfad ohne ConfigMap
<code>oid4vp.trust.x5cAnchorsPath</code>	<code>OID4VP_X5C_TRUST_ANCHORS_PATH</code>

Ohne `oid4vp.trust.enabled` kennt die Instanz kein Vertrauensdokument und weist jede vorgelegte Credential ab. Der vorgesehene Weg ist die eigene ConfigMap. Sie wird nicht von Helm verwaltet und überlebt damit eine Deinstallation des Release; die Vertrauensentscheidungen einer Instanz hängen so nicht am Lebenszyklus ihres Deployments. Bleibt `existingConfigMap` leer, greift `dataPath` und zeigt auf das im Image mitgelieferte Dev-Fixture. Warum das einen Render-Abbruch auslöst und wo die Ausnahme liegt, steht in A7.4.

`x5cAnchorsPath` benennt die Vertrauensanker, gegen die eine Credential geprüft wird, die ihre Schlüssel als x5c-Kette mitbringt. Bleibt der Pfad leer, wird eine solche Credential abgewiesen. Die Instanz glaubt nie dem Blattzertifikat, das die Credential selbst mitliefert.

Gegen dieselben Anker prüft die Instanz auch signierte Statuslisten, die ihre Schlüssel als x5c-Kette im Token-Header mitbringen; das Blattzertifikat muss dabei den Issuer identifizieren, den das Token nennt. Eine unsignierte Statusliste wird in keiner Konfiguration akzeptiert (ADR-34); einen Schalter, der das erlaubt, gibt es im Backend nicht. Das Anker-Bündel muss deshalb die Roots **aller** Issuer enthalten, deren Credentials oder Statuslisten die Instanz prüft, nicht nur den des PID-Issuers (A7.2).

Der inhaltliche Aufbau des Vertrauensdokuments (Zwecke, Organisationen, Mechanismen) steht in A7.3.

Anbindung an den Federated Catalogue

Wert	Gerenderte Wirkung
<code>federatedCatalogue.enabled</code>	Der gesamte FC-Konfigurationsblock
<code>federatedCatalogue.apiUrl</code> bzw. <code>.secretRef</code>	<code>FEDERATED_CATALOGUE_API_URL</code>
<code>federatedCatalogue.oauth.clientID</code>	<code>FEDERATED_CATALOGUE_CLIENT_ID</code>
<code>federatedCatalogue.oauth.clientSecretSecretRef</code> bzw. <code>.clientSecret</code>	<code>FEDERATED_CATALOGUE_CLIENT_SECRET</code>
<code>fcKeycloak.realmURL</code>	<code>FC_KEYCLOAK_REALM_URL</code>
<code>federatedCatalogue.remote.acknowledgeAdminAllTrustBoundary</code>	Freigabe des Renders bei entferntem Katalog

Der Katalog-Block ist **alles-oder-nichts**: Sobald einer der vier Werte API-URL, Client-ID, Client-Secret und Keycloak-Realm-URL gesetzt ist, fordert das Backend beim Start alle vier. Fehlt einer, benennt die Abbruchmeldung ihn.

Drei Werte sind die üblichen Fehlerquellen:

- **`apiURL` muss den Servlet-Kontextpfad des fc-service enthalten.** Eine öffentliche URL trägt ihn meist schon, eine In-Cluster-Adresse nicht. Ohne ihn antwortet der Katalog auf jeden Aufruf mit 404 (A13.3).
- **`oauth.clientID` muss ein Client sein, den der verwendete Realm wirklich anlegt.** Der mitgelieferte Realm legt `federated-catalogue` an; die Chart-Vorgabe `dcs-fc-client` passt dazu nicht und quittiert gegen den mitgelieferten Katalog mit 401.
- **`fcKeycloak.realmURL` ist die öffentliche Realm-URL.** fc-service prüft den `iss`-Claim des Tokens dagegen; ein über eine In-Cluster-Adresse geprägtes Token wird abgelehnt. Der mitgelieferte Realm heißt `federated-catalogue-realm` und liegt unter der Basis `/auth`; die URL hat damit die Form `https://<host>/auth/realms/federated-catalogue-realm`.

`acknowledgeAdminAllTrustBoundary` ist zwingend `true`, wenn `federatedCatalogue.enabled` ohne `fcservice.enabled` gesetzt ist, also gegen einen entfernten Katalog. Der Dienstzugang zum Katalog trägt `ADMIN_ALL`. Jede Instanz an diesem Katalog kann damit jeden Eintrag darin verwalten, auch die der anderen. Ein geteilter Katalog ist deshalb nur innerhalb einer gemeinsam vertrauenden Verwaltungsgrenze zulässig, und die Bestätigung erzwingt, dass jemand diese Entscheidung bewusst trifft.

Mitgelieferter Katalog

Wert	Gerenderte Wirkung
<code>fcservice.enabled</code>	Katalog, Keycloak, Fuseki und fc-postgres im Release
<code>fcservice.realmProvision.enabled</code>	Realm-Import-Hook
<code>fcservice.keycloak.hostname</code>	Issuer-URL, die fc-service erwartet
<code>fcKeycloakApp.hostname</code>	Öffentlicher Hostname des Keycloak
<code>fcservice.portal.enabled</code>	Demo-Portal des Katalogs
<code>fcservice.route.path</code>	Ingress-Regel auf <code>fc-service:8081</code>
<code>fcKeycloakApp.auth.adminUser / .adminPassword</code>	Admin-Zugang des mitgelieferten Keycloak, als Secret
<code>fcservice.realmProvision.adminUser / .adminPassword</code>	Zugang, mit dem der Import-Hook den Realm anlegt

Diese Werte betreffen nur den mitgelieferten Katalog (A1.1). Der Realm-Import ist bei einer frischen Katalog-Datenbank erforderlich; er läuft über die Keycloak-Admin-API. `fcservice.keycloak.hostname` und `fcKeycloakApp.hostname` müssen denselben öffentlichen Hostnamen tragen. Das ist Handarbeit, weil eine Werte-Datei sich nicht selbst referenzieren kann. Bleibt `fcservice.keycloak.hostname` leer, bildet fc-service die Issuer-URL aus dem In-Cluster-Service; hinter einem echten Ingress ist das falsch und führt zu den 401-Bildern aus A13.3. `fcservice.route.path` veröffentlicht den Katalog über den Ingress, damit ihn eine Instanz aus einem anderen Cluster erreichen kann.

`fcservice.portal.enabled` schaltet die Weboberfläche des Katalogs zu. Sie ist per Vorgabe **an** und dient der Demonstration. Das DCS braucht sie nicht, es spricht mit dem Katalog über dessen API. Wo Ressourcen knapp sind oder die veröffentlichte Oberfläche klein bleiben soll, kann sie aus.

Die beiden Keycloak-Admin-Zugänge tragen in der Chart-Vorgabe `admin / admin` und sind vor jedem erreichbaren Deployment zu überschreiben. Der eine ist der Administrator des mitgelieferten Keycloak, der andere der Zugang, mit dem der Import-Hook den Realm anlegt. Beide müssen denselben Wert tragen, sonst scheitert der Import an der Anmeldung. Setze den Wert über `--set` aus dem Secret-Manager, nicht in einer committeten Werte-Datei. Der Import-Hook rendert ihn als Klartext in seine Job-Spezifikation; wer im Namespace `get job` darf, liest ihn dort mit. Das gilt bis zur Umsetzung derselben `secretKeyRef`-Änderung wie beim System-Token (A6.6).

A5.6 Abhängige Dienste und Callbacks

Wert	Gerenderte Wirkung	Wirkung im System
<code>dss.enabled</code> bzw. <code>dss.url</code>	<code>DSS_URL</code>	Der Validator für extern erzeugte Signaturen. Ohne ihn weist die Instanz jede Signatur ab; Vertragserstellung funktioniert weiter, der Ausfall zeigt sich erst beim Signieren. Ist <code>DSS_URL</code> gesetzt, die Instanz aber un erreichbar, ist die Validierung ein harter Fehler.
<code>statuslistService.enabled</code> bzw. <code>.url</code>	<code>STATUSLIST_SERVICE_URL</code>	Beim Start zwingend, und der Dienst wird beim Start aktiv angefragt
<code>statuslistService.tenantID</code>	<code>STATUSLIST_TENANT_ID</code>	Leer heißt <code>default</code> . Zwei Instanzen, die sich einen Status-Listen-Dienst teilen, brauchen verschiedene Tenants, sonst vergeben sie dasselbe Widerrufsbit an zwei Verträge.
kein Chart-Wert, nur über <code>extraEnv</code>	<code>STATUSLIST_LIST_ID</code>	Die Liste, in der neue Widerrufseinträge vergeben werden; Vorgabe ist Liste 1. Gesetzt wird sie erst, wenn Liste 1 voll ist, und erst nachdem der Betreiber die Nachfolgeliste im Status-List-Dienst angelegt und in <code>status_list_cursors</code> registriert hat. Bei einer nicht registrierten Liste schlägt die Vergabe hart fehl, statt einen Eintrag auszugeben, den niemand ausliefert. Ein nicht-positiver Wert bricht den Start ab.
<code>pdfCore.enabled</code> bzw. <code>pdfCore.url</code>	<code>PDF_CORE_URL</code>	Beim Start zwingend
<code>pdfCore.ontologyBaseURL</code>	<code>DCS_PDF_CORE_ONTOLOGY_BASE_URL</code> im pdf-core-Container	Basis der Ontologie-IRIs in Payloads und im ausgelieferten JSON-LD-Kontext. Leer heißt: der In-Cluster-Service-URL. Hinter einer öffentlichen Adresse ist diese zu setzen.
<code>pdfCore.replicaCount</code> , <code>.resources</code> , <code>.service.port</code>	pdf-core-Deployment	Dimensionierung des Renderers. Er ist der Engpass bei Massenerzeugung und beim erneuten Rendern eingehender Dokumente zur Gegenprüfung
<code>signing.tsaURL</code>	<code>TSA_URL</code>	Siehe A5.4 für die Wirkung und A5.8 dazu, worauf diese URL per Vorgabe zeigt
<code>archiveNotary.baseURL</code> , <code>.notaryPath</code> , <code>.auditLogPath</code>	<code>ORCE_ARCHIVE_NOTARY_URL</code> , <code>ORCE_ARCHIVE_AUDIT_LOG_URL</code>	Leere <code>baseURL</code> und <code>orice.enabled</code> ergeben <code>http://<release>-orice:<orice-port></code>
<code>global.archiveAuditTokenSecretRef</code> bzw. <code>global.archiveAuditToken</code>	<code>ORCE_ARCHIVE_AUDIT_LOG_BEARER_TOKEN</code>	Ohne Token schlägt die Archiv-Integritätsprüfung fehl
<code>auditExecutor.url</code> , <code>.path</code> , <code>.timeout</code>	<code>PAC_AUDIT_EXECUTOR_URL</code> , <code>PAC_AUDIT_EXECUTOR_TIMEOUT</code>	Leere <code>url</code> und <code>orice.enabled</code> ergeben den mitgelieferten ORCE-Endpunkt. Ein Kunde ersetzt den Executor rein über Konfiguration.
<code>global.auditExecutorTokenSecretRef</code> bzw. <code>.auditExecutorToken</code>	<code>PAC_AUDIT_EXECUTOR_BEARER_TOKEN</code>	Nur gerendert, wenn eines von beiden gesetzt ist
<code>workflowGateExecutor.url</code> , <code>.path</code> , <code>.timeout</code>	<code>PAC_WORKFLOW_GATE_EXECUTOR_URL</code> , <code>PAC_WORKFLOW_GATE_EXECUTOR_TIMEOUT</code>	Ein ungültiger Timeout-Wert bricht den Start ab
<code>global.workflowGateTokenSecretRef</code> bzw. <code>.workflowGateToken</code>	<code>PAC_WORKFLOW_GATE_EXECUTOR_BEARER_TOKEN</code>	Nur gerendert, wenn eines von beiden gesetzt ist. Ohne Token wird der Gate-Aufruf nicht autorisiert
<code>orice.checkpointSink.enabled</code> , <code>.url</code> , <code>.timeoutSeconds</code> , <code>.internalTimeoutSeconds</code> , <code>.bearerToken</code> bzw. <code>.bearerTokenSecretRef</code> , <code>.allowInsecureTLS</code>	<code>CHECKPOINT_SINK_*</code> im ORCE-Container	Publiziert Audit-Checkpoints in Reihenfolge an einen externen HTTPS-Sink. Einschaltet verweigert das Chart den Render ohne HTTPS-URL, ohne positiven internen Timeout oder ohne Bearer-Token. Ein vom Sink gemeldeter <code>sequence_gap</code> oder <code>previous_root_mismatch</code> blockiert die Publikation dauerhaft und ist als Integritäts-Incident zu behandeln; das Verfahren steht in <code>docs/audit-checkpoint-workflow-gate-operations.md</code> . <code>allowInsecureTLS</code> wirkt nur zusammen mit den Test-Controls und bleibt in Produktion aus.
<code>contractTargets</code>	ConfigMap plus <code>CONTRACT_TARGETS_FILE = /etc/dcs/contract-targets/targets.json</code>	Startbestand der Zielsystem-Registry. Seeding ist anlegend, nicht abgleichend , gematcht über <code>name</code> : ein in der Oberfläche umgehängter Eintrag wird beim nächsten Start nicht zurückgesetzt, und ein hier entfernter Eintrag verschwindet nicht aus einer laufenden Instanz.

Wert	Gerenderte Wirkung	Wirkung im System
<code>sharedServices.enabled</code> , <code>.host</code> , <code>.services[]</code> , <code>.ingressClassName</code> , <code>.tls</code> , <code>.proxyBodySize</code> , <code>.annotations</code>	Ingress unter <code>https://<host>/shared/<name></code>	Veröffentlicht die zustandslosen Dienste dieser Instanz für einen Peer, der keine eigenen betreiben kann. Das verringert die Unabhängigkeit dieses Peers und ist eine bewusste Entscheidung.
<code>acmeRenewal.enabled</code> , <code>.host</code> , <code>.email</code> , <code>.secretName</code> , <code>.server</code> , <code>.schedule</code> , <code>.image</code> , <code>.challengePort</code> , <code>.rbac.create</code> , <code>.serviceAccountName</code> , <code>.ingressClassName</code> , <code>.ingressAnnotations</code> , <code>.resources</code> , <code>.persistence</code>	CronJob, Service, Ingress und optional RBAC für eine HTTP-01-Erneuerung	Fallback für Cluster, in denen cert-manager den Solver-Pod nicht schedulen kann. <code>host</code> , <code>email</code> und <code>secretName</code> sind Pflicht, sonst bricht der Render ab. Wo cert-manager funktioniert, bleibt das aus.
<code>complianceMonitor.enabled</code> , <code>.schedule</code> , <code>.startingDeadlineSeconds</code> , <code>.timeout</code> , <code>.resources</code>	CronJob <code><fullname>-pac-monitor</code>	Führt den Compliance-Durchlauf planmäßig statt auf Anfrage. Vorgabe alle fünf Minuten mit <code>concurrencyPolicy: Forbid</code> . Ausgeschaltet werden Risiken nur noch gefunden, wenn jemand fragt.

Der Status-Listen-Dienst und ADR-34

An dieser Stelle laufen Chart und Architekturentscheidung auseinander, und beides gehört gesagt. ADR-34 nimmt den XFSC-Status-Listen-Dienst aus dem Projekt: Er lieferte in jeder hiesigen Konfiguration unsigned Listen aus, und eine unsigned Statusliste behauptet einen Widerrufsstatus, für den niemand bürgt. Beschlossen ist, dass jeder Issuer seine Statusliste selbst signiert und selbst ausliefert und dass eine unsigned Liste nirgends mehr akzeptiert wird.

Umgesetzt ist davon die prüfende Seite: Das Backend kennt keinen Schalter, der unsigned Listen zulässt, und die mitgelieferten Issuer liefern ihre Statuslisten signiert selbst aus (A7). Die betreibende Seite ist noch nicht nachgezogen: Das Subchart liegt weiter im Chart, die BDD- und Dev-Werte betreiben den Dienst weiter, und das Backend verlangt `STATUSLIST_SERVICE_URL` unverändert als Startgatter, weil es die Widerrufseinträge seiner Vertragsdokumente dort vergibt. Die drei Zeilen oben beschreiben diesen Ist-Zustand.

Bis Chart und Backend dem ADR folgen, gilt: Der Dienst bleibt eine Startvoraussetzung, seine Listen sind aber unsigned und werden von jedem konformen Verifier abgewiesen, dem eigenen eingeschlossen. Ein Widerruf, der nur in einer solchen Liste steht, ist für niemanden prüfbar.

A5.7 Plattform und Betriebsmechanik

Wert	Gerenderte Wirkung	Wirkung im System
<code>replicaCount</code>	Replikate des Backends	<code>autoscaling.enabled</code> übersteuert das Feld
<code>autoscaling.enabled</code> , <code>.minReplicas</code> , <code>.maxReplicas</code> , <code>.targetCPUUtilizationPercentage</code>	HorizontalPodAutoscaler	Eingeschaltet ersetzt der HPA <code>replicaCount</code> als Quelle der Replikatzahl
<code>readinessProbe</code>	Readiness-Probe auf <code>/readyz</code>	Absichtlich von der Prozess-Lebendigkeit getrennt. Siehe A12.1.
<code>livenessProbe</code>	Liveness-Probe am Backend-Container, sobald der Wert gesetzt ist	Per Vorgabe nicht gesetzt , der Backend-Container hat also keine Liveness-Probe. Das passt zum Bootstrap-Verhalten aus A12.1: Ein Pod, der minutenlang auf sein Token wartet, ist nicht tot. Wer eine will, setzt hier eine, die diese Wartezeit aushält.
<code>resources</code>	Ressourcenblock des Backend-Containers	Ohne Block greift auf Namespaces mit LimitRange der dortige Default
<code>podSecurityContext</code> , <code>securityContext</code> , <code>nodeSelector</code> , <code>tolerations</code> , <code>affinity</code> , <code>podAnnotations</code> , <code>podLabels</code> , <code>hostAliases</code>	Direkt in das Deployment übernommen	Die üblichen Platzierungs- und Härtnungsknöpfe; das Chart wertet sie nicht aus, sondern reicht sie durch
<code>volumes</code> , <code>volumeMounts</code>	Zusätzliche Volumes am Backend	Der Weg, um etwa den PID-Root-CA für <code>oid4vp.trust.x5cAnchorsPath</code> einzuhängen
<code>extraEnv</code> , <code>extraEnvFrom</code>	Zusätzliche Umgebungsvariablen	<code>extraEnvFrom</code> schaltet zugleich die Render-Zeit-Prüfung des Dev-Fixtures ab, weil deren Inhalt beim Rendern nicht lesbar ist
<code>customCA.enabled</code> , <code>.configMapName</code>	Mount nach <code>/usr/local/share/ca-certificates/custom</code>	Für TLS-Verbindungen gegen eine eigene CA
<code>serviceAccount.create</code> , <code>.name</code> , <code>.annotations</code> , <code>.automount</code>	ServiceAccount des Backends	Bei aktivem FC-Realm-Provisioning wird stattdessen der <code>-fc-lifecycle</code> -Account verwendet, weil das Backend dann auf den Hook-Job wartet
<code>monitoring.enabled</code> und die Blöcke <code>grafana</code> , <code>prometheus</code> , <code>alertmanager</code> , <code>nodeExporter</code> , <code>kubeStateMetrics</code>	kube-prometheus-stack als Subchart	Nur nötig, wenn die Instanz ihr Monitoring selbst mitbringen soll. Die mitgelieferten Beispielwerte verwenden <code>storageClassName: local-path</code> ; in einem Cluster ohne diese Klasse bleiben die PVCs ungebunden. Grafana ist per Vorgabe zusätzlich abgeschaltet; wer es einschaltet, überschreibt <code>grafana.adminPassword</code> , weil dort ein Platzhalter steht.
<code>istio.enabled</code> , <code>.hosts</code> , <code>.gateways</code>	VirtualService statt Ingress-Objekt	Für Cluster, in denen der Ingress über ein Service-Mesh-Gateway läuft. <code>hosts</code> und <code>gateways</code> müssen zu den vorhandenen Gateway-Objekten passen; das Chart legt keine an.

A5.8 Die Zeitstempel-Autorität ist per Vorgabe lokal

`TSA_URL` ist ein Startgatter. Die naheliegende Annahme, die Instanz spreche mit einer externen Zeitstempel-Autorität, trifft in der Vorgabekonfiguration **nicht** zu.

Das ORCE-Subchart setzt `localTSA.enabled: true`. Der `/tsa`-Flow signiert Zeitstempelantworten im Pod selbst mit einem Zertifikat, das der `localTSA.autoProvision`-Hook erzeugt und im Secret `<release>-orce-tsa-material` (Schlüssel `tsa-cert.pem`, `tsa-key.pem`) ablegt. Es geht kein Verkehr nach außen, und der Seriennummernzähler liegt unter `/data/tsa-state/serial` auf dem ORCE-Volume.

Betriebliche Folgen:

- Ein Zeitstempel aus der Vorgabekonfiguration ist ein **Entwicklungs-Zeitstempel**. Er belegt nichts gegenüber Dritten. Wer RFC-3161-Belege mit Beweiswert braucht, wechselt auf eine echte Autorität.
- Der Wechsel ist am Backend eine reine Konfigurationsänderung und erfordert **keinen Neubau des Backends**. Das im Binary eingebettete Zertifikat ist nur der Vorgabewert, `TSA_TRUST_CERT_FILE` übersteuert ihn.
- Wird `localTSA` abgeschaltet, entfällt auch das automatisch bereitgestellte Secret. Zeigt `signing.tsaTrustCertSecret` weiter darauf, schlägt der Mount fehl.

Der Weg zum belastbaren Zeitstempel

Für die Anbindung einer echten Autorität gibt es zwei Wege. Beide enden bei denselben zwei Werten am Backend: `signing.tsaURL` auf die Autorität und `signing.tsaTrustCertSecret` auf deren CA-Zertifikat (Schlüssel `tsa-cert.pem`).

- Direkt.** `signing.tsaURL` zeigt auf den RFC-3161-Endpunkt der Autorität. Das ist der kürzeste Weg. Er passt, wenn die Instanz die Autorität selbst erreichen darf.

2. **Über ORCE.** ORCE ist eine Node-RED-basierte Low-Code-Umgebung, und der `/tsa`-Endpunkt ist ein Flow darin. Dieser Flow lässt sich durch einen ersetzen, der die Zeitstempelanfrage an eine kommerzielle Autorität weiterreicht, etwa DigiCert, und deren Antwort zurückgibt. `signing.tsaURL` bleibt dann auf ORCE gerichtet. **Dies ist der vorgesehene Weg vom Entwicklungs-Zeitstempel zum belastbaren.** Zugangsdaten, Kontingente und ein Ausweichziel lassen sich dort ohne Code-Änderung pflegen, und die Instanzen einer Föderation sehen weiterhin dieselbe Endpunktadresse.

Ein solcher vermittelnder Flow gehört **nicht** zur Auslieferung. Das Chart bringt nur den lokalen Entwicklungs-Flow mit. Der Betreiber legt den vermittelnden Flow in ORCE selbst an und schaltet dabei `localTSA.enabled` ab, damit nicht weiter im Pod signiert wird.

Verifikation, dass der Zeitstempel nicht mehr lokal entsteht: Das Zertifikat, gegen das das Backend Zeitstempel prüft, kommt aus `signing.tsaTrustCertSecret`. Nach der Umstellung muss dieses Secret das CA-Zertifikat der externen Autorität tragen und darf nicht mehr `<release>-orce-tsa-material` sein:

```
kubectl -n <ns> get deployment/<release>-digital-contracting-service \
-o jsonpath='{.spec.template.spec.volumes[?(@.name=="tsa-trust")].secret.secretName}'{"\n"}
```

Ein neu erzeugter Vertrag trägt danach einen Zeitstempel, dessen Ausstellernamen die externe Autorität nennt. Solange dort der ORCE-Secret-Name steht, prüft die Instanz weiter gegen das selbstsignierte Entwicklungsmaterial.

A6 Identitäten, Secrets und HSM

A6.1 Die Schlüssel einer Instanz

Jeder private Schlüssel einer DCS-Instanz liegt in einem PKCS#11-Token (DCS-IR-HI-01). Es gibt keinen Software-Fallback. Fünf Schlüssel, alle ECDSA P-256:

Label	Zweck	Besonderheit
<code>dc-did</code>	Signiert die Identitätsnachweise dieser Instanz (JAdES, DCS-zu-DCS-Challenge)	Muss zum veröffentlichten <code>did.json</code> passen, sonst bricht der Start ab
<code>dc-vc</code>	Signiert Lifecycle- und Federation-Credentials	Keine
<code>dc-oid4vp-jar</code>	Signiert OpenID4VP-Request-Objekte	Keine
<code>dc-c2pa</code>	Signiert C2PA-Claims	Trägt die x5chain, die pdf-core in den COSE-Header einbettet
<code>dc-ecdh</code>	Wrappt Content-Encryption-Keys	Braucht <code>CKA_DERIVE</code> . Beim Start läuft ein Wrap/Unwrap-Selbsttest gegen den <code>keyAgreement</code> -Eintrag im <code>did.json</code> ; schlägt er fehl, startet die Instanz nicht.

Die Labels sind über `DCS_HSM_KEY_DID`, `DCS_HSM_KEY_VC`, `DCS_HSM_KEY_JAR`, `DCS_HSM_KEY_C2PA` und `DCS_HSM_KEY_ECDH` überschreibbar (per `extraEnv`).

Es gibt keinen Vertragssignaturschlüssel. Eine Vertragssignatur entsteht in der Wallet der unterzeichnenden Person. Das DCS prüft sie.

Der Grund ist rechtlich, nicht technisch. Artikel 26 der eIDAS-Verordnung (EU) 910/2014 verlangt für eine fortgeschrittene elektronische Signatur, dass sie eindeutig dem Unterzeichner zugeordnet ist, ihn identifiziert und mit Signaturerstellungsdaten erzeugt wird, die er unter seiner alleinigen Kontrolle verwenden kann. Ein Schlüssel im HSM des DCS, mit dem die Instanz für alle Unterzeichnenden signiert, erfüllt die alleinige Kontrolle nicht. Die Herleitung steht in ADR-12; ADR-1 nimmt die Vertragssignatur aus demselben Grund aus der Liste der DCS-Schlüssel heraus.

A6.2 SoftHSM2 im Cluster (Dev, Staging, CI)

Mit `pkcs11.provisioning.enabled=true` rendert das Chart:

- einen PVC `<fullname>-hsm-token`,
- ein Secret `<fullname>-hsm-pin` (falls kein `pinSecretRef` gesetzt ist),
- einen `post-install,post-upgrade`-Hook-Job `<fullname>-hsm-provision`.

Der Job initialisiert den Token, erzeugt die fünf Schlüssel, stellt die C2PA-x5chain gegen eine im Token-Verzeichnis liegende Dev-CA aus und leitet `did.json` mit `/app/gendid` aus dem `dc-did`-Schlüssel ab. Er ist idempotent und läuft bei jedem Install und Upgrade.

SoftHSM2 ist ein Softwaretoken und **kein Produktions-HSM**.

A6.3 Echtes HSM (Produktion)

Achtung, das ist nicht in jedem Fall reine Konfiguration. Am Backend sind Modulpfad, Token-Label und PIN Werte. Das Herstellermodul selbst muss aber in den Container kommen, und je nach Hersteller reicht ein zusätzliches Volume dafür nicht: Bringt das Modul eigene Bibliotheken, eine Konfigurationsdatei an fester Stelle, einen Client-Daemon oder eine Lizenzdatei mit, ist ein **Neubau des Container-Images** nötig, der diese Bestandteile enthält. Dazu können **hardwarespezifische Anpassungen am Deployment** kommen: der Zugriff auf ein PCIe- oder USB-Gerät über einen Device-Plugin, ein `hostPath` oder erweiterte Container-Rechte, eine Netzwerkfreigabe zum Netzwerk-HSM, ein eigener Init-Container für die Client-Registrierung. Plane den Wechsel auf ein Hersteller-HSM deshalb als Änderung an Image und Deployment, nicht als Werte-Änderung. Welche Bestandteile konkret nötig sind, steht in der Integrationsdokumentation des HSM-Herstellers; das DCS-Repository enthält keine herstellerspezifische Vorlage.

Voraussetzungen:

1. Das PKCS#11-Modul des Herstellers ist im Backend-Container erreichbar, über ein zusätzliches Volume aus `volumes / volumeMounts` oder über ein abgeleitetes Image (siehe Warnhinweis oben).
2. Alle fünf Schlüssel existieren bereits im Token, `dcS-ecdh` mit `CKA_DERIVE`. Das Chart legt in Produktion keine HSM-Schlüssel an.
3. Ein Secret mit der Token-PIN existiert im Ziel-Namespace.
4. Ein Secret mit der C2PA-Zertifikatskette existiert im Ziel-Namespace (Schlüssel `x5chain-pem`, sofern nicht per `pdfCore.signing.existingSecretX5ChainKey` anders benannt). Ohne den Provisionierungs-Hook erzeugt niemand dieses Secret.
5. Ein `did.json`, das genau die Token-Schlüssel veröffentlicht, ist als Secret vorhanden und über `identity.secretName` referenziert. Es muss einen `keyAgreement`-Eintrag auf `dcS-ecdh` tragen.

Werte:

```
pkcs11:
  modulePath: /usr/lib/<vendor>/libpkcs11.so
  tokenLabel: <token-label>
  pinSecretRef:
    name: <pin-secret>
    key: PKCS11_PIN
  provisioning:
    enabled: false

pdfCore:
  signing:
    existingSecret: <x5chain-secret>

identity:
  enabled: true
  secretName: <did-secret>
```

Verifikation: Nach dem Rollout darf im Backend-Log keine Zeile `waiting for PKCS#11 token` mehr erscheinen, und es muss `HTTP server listening` folgen:

```
kubectl -n <ns> logs deployment/<release>-digital-contracting-service --tail=200 | grep -E "waiting for PKCS#11 token|HTTP server listening"
```

Fehlerbilder: siehe A13.

A6.4 Secret-Inventar

In diesem Dokument steht kein Secret-Wert. Aufgeführt sind Herkunft, Verbraucher und die Wirkung des Fehlens.

Secret	Herkunft	Verbraucher	Wirkung ohne
<fullname>-hsm-pin (oder pkcs11.pinSecretRef)	Chart aus pkcs11.pin, oder vorab angelegt	Backend, HSM-Provisionierungs-Job	Kein Token-Zugriff, Backend bleibt im Bootstrap
<fullname>-hsm-c2pa-x5chain	HSM-Provisionierungs-Hook (nur mit publishSecrets=true)	pdf-core	pdf-core bleibt im Init-Container wait-for-x5chain
<fullname>-identity	HSM-Provisionierungs-Hook (nur mit publishSecrets=true)	Backend als DCS_DID	Backend liest kein DID-Dokument und beendet sich
pdfCore.signing.existingSecret	Betreiber, aus der HSM-ausgestellten x5chain	pdf-core	Nur im Produktionsmodus relevant; pdf-core startet nicht
database.secretRef	Betreiber bzw. Secret-Manager	Backend, pac-monitor-CronJob	Keine Datenbankverbindung
messaging.secretRef	Betreiber	Backend	Kein Event-Bus
federatedCatalogue.secretRef, federatedCatalogue.oauth.clientSecretSecretRef	Betreiber	Backend	Katalog-Anmeldung schlägt fehl; der Start bricht ab
ipfs.tenantBaseURLSecretRef	Betreiber	Backend	IPFS-Konfiguration unvollständig, Start bricht ab
global.archiveAuditTokenSecretRef	Betreiber	Backend	Archiv-Integritätsprüfung nicht möglich
global.auditExecutorTokenSecretRef, global.workflowGateTokenSecretRef	Betreiber	Backend	Der jeweilige Executor-Aufruf wird nicht autorisiert
orce.checkpointSink.bearerTokenSecretRef (oder .bearerToken)	Betreiber	ORCE	Bei aktiviertem Checkpoint-Sink verweigert das Chart ohne Token den Render (A5.6)
signing.tsaTrustCertSecret (Schlüssel tsa-cert.pem)	Betreiber bzw. ORCE-TSA-Hook	Backend	Es gilt das im Binary eingebettete TSA-Zertifikat
<release>-orce-tsa-material (Schlüssel tsa-cert.pem, tsa-key.pem)	ORCE-Subchart-Hook (localTSA.autoProvision) oder vorab angelegt	ORCE	Der lokale TSA-Flow signiert nicht
Hydra-Systemsecret und Client-Secrets	Betreiber über --set	Hydra	Diese Chart-Version hat dafür keine secretKeyRef
fc-keycloak (Schlüssel admin-user, admin-password)	Chart aus fcKeycloakApp.auth.*	Mitgeliefertes Keycloak	Kein Admin-Zugang zum Katalog-Keycloak. Die Chart-Vorgabe ist admin / admin und vor jedem erreichbaren Deployment zu ersetzen (A5.5). Der Name ist fest und nicht vom Release abgeleitet: zwei Releases im selben Namespace kollidieren darauf
DCS_SYSTEM_TOKEN (Wert aus systemToken)	Betreiber über --set	Backend	Das Deployment rendert nicht; siehe A6.6
imagePullSecrets bzw. dockerRegistry.*	Betreiber	Alle Pods	Images werden nicht gezogen

A6.5 Betrieb ohne RBAC-Rechte

Darf die installierende Identität im Namespace keine roles / rolebindings anlegen, schlägt der Install mit cannot create resource roles fehl. Dann:

```
pkcs11:
  provisioning:
    publishSecrets: false
orce:
  localTSA:
    autoProvision: false
acmeRenewal:
  rbac:
    create: false
  serviceAccountName: <bestehender-account>
```

Konsequenzen, die dazugehören:

- Der Provisionierungs-Hook schreibt nichts über die Kubernetes-API. did.json und c2pa-x5chain.pem bleiben auf dem gemeinsamen Token-Volume; das Backend liest DCS_DID von dort, pdf-core mountet dasselbe Volume readOnly.

- Weil ein `ReadWriteOnce`-Volume mehrere Pods nur auf einem Node zulässt, wird pdf-core in diesem Modus per `podAffinity` auf den Node des Backends gepinnt. Wer das nicht will, setzt `pkcs11.provisioning.persistence.accessModes: ["ReadWriteMany"]`.
- Das TSA-Material-Secret muss vorab angelegt werden. Es ist selbstsigniert und hängt an nichts im Cluster.

A6.6 Das System-Token

`systemToken` ist das In-Cluster-Dienstcredential, mit dem sich die Hintergrund-PDF-Regeneration gegen die internen Signaturprimitive authentifiziert. Sie läuft auf NATS-Ereignissen und hat kein Benutzer-JWT.

Zwei Eigenschaften, die zusammen zählen:

- Das Backend-Template fordert den Wert per `required` an. Ein Deployment ohne ihn rendert nicht. Einen im Repository mitgelieferten Vorgabewert gibt es damit nicht.
- Die Prüfung im Backend schränkt den Träger des Wertes nicht ein. Wer ihn besitzt, ist auf jeder authentifizierten Route ein Aufrufer mit voller Berechtigung. Dass er nur im Cluster zirkuliert, sichert das Deployment zu, nicht die Prüfung.

Daraus folgen drei Regeln für den Betrieb:

1. **Pro Instanz ein eigener, zufälliger Wert.** Die Werte-Dateien für Entwicklung und Tests tragen alle dasselbe Literal. Das ist ein Testcredential und darf in keinem erreichbaren Deployment auftauchen.
2. **Injektion über `--set` aus dem Secret-Manager**, nicht über eine committete Werte-Datei.
3. **Rotation wie ein Passwort behandeln:** Wert ändern, `helm upgrade`, alte Pods durchrollen lassen.

Offene Härtungsmassnahme: `secretKeyRef` für `DCS_SYSTEM_TOKEN`

Das Chart rendert den Wert derzeit als Klartext-`value` in das Backend-Deployment. Wer im Namespace `get deployment` darf, liest ihn damit mit, auch ohne Leserecht auf Secrets. Das ist behebbar, und zwar im Chart:

1. Im Backend-Deployment-Template die Umgebungsvariable `DCS_SYSTEM_TOKEN` aus einer `secretKeyRef` beziehen statt aus einem `value`.
2. Einen Werte-Pfad `systemTokenSecretRef` mit `name` und `key` einführen und die bestehende `required`-Prüfung so erweitern, dass entweder `systemToken` oder `systemTokenSecretRef.name` gesetzt sein muss. Damit bleibt ein Deployment ohne Token weiterhin nicht renderbar.
3. Für Produktion das Secret vorab anlegen und nur noch die Referenz in der Werte-Datei führen.

Verifikation nach der Umsetzung: Der folgende Aufruf darf keinen Klartextwert mehr ausgeben, sondern `valueFrom`:

```
kubectl -n <ns> get deployment/<release>-digital-contracting-service \
-o jsonpath='{.spec.template.spec.containers[0].env[?(@.name=="DCS_SYSTEM_TOKEN")]}{"\n"}'
```

Solange das Chart diesen Weg nicht kennt, bleibt es bei Regel 2 oben: den Wert über `--set` aus dem Secret-Manager injizieren und Leserechte auf Deployments im Namespace so eng halten wie Leserechte auf Secrets.

A7 Credential-Issuer und Trust-Dokument

Der mitgelieferte Issuer ist ein DEMO-Credential-Issuer. Einen echten Credential-Issuer liefert das DCS nicht.

Das gilt für beide Sorten. Der mitgelieferte Vollmachts-Issuer stellt eine Credential mit dem Anzeigenamen „FACIS Demo Power of Attorney“ aus, ohne die Vollmacht zu prüfen. Der mitgelieferte PID-Issuer stellt einen Identitätsnachweis aus, ohne die Person zu prüfen; seine Credential ist keine EUDI-PID und darf nicht als solche präsentiert werden. Beide erzeugen ihr Schlüsselmaterial beim ersten Start selbst und sind an keine Vertrauensinfrastruktur angeschlossen.

Für einen Betrieb mit Rechtswirkung stellt der Betreiber die Issuer selbst: den Vollmachts-Issuer aus seinem eigenen Berechtigungswesen, den Identitätsnachweis von einem Herausgeber, der Personen tatsächlich prüft. Beide binden sich über das Vertrauensdokument (A7.3) an eine Instanz. Das DCS schreibt nicht vor, welche Produkte das sind; es verlangt nur, dass sie OID4VCI sprechen und über einen der in A7.3 genannten Mechanismen prüfbar sind.

Ein `dc`s-Release enthält **keinen** Credential-Issuer. Die beiden Credentials, die eine unterzeichnende Person vorlegt, sind die Vollmacht (Power of Attorney) und der Personenidentitätsnachweis (PID). Beide kommen aus OID4VCI-Issuern, die als eigene Helm-Releases des `orce`-Subcharts laufen, jeweils mit eigenem Volume, eigenem Schlüssel und eigener DID.

Release	Werte-Datei	Stellt aus	Anzahl
<code>dc</code> s-issuer	<code>values.issuer.yml</code>	Demo-Vollmacht	Einer je DCS-Instanz, unter deren Hostnamen
<code>dc</code> s-issuer	<code>values.issuer2.yml</code>	Demo-Vollmacht	Für die zweite Instanz, in deren Namespace
<code>dc</code> s-pid-issuer	<code>values.pid-issuer.yml</code>	Demo-PID	Einmal für die gesamte Föderation

`flowsDir` wählt den Flow-Satz eines Release: `flows` ist der DCS-seitige Satz (Trust-PDP, Archiv, TSA, Contract-Target), `flows-issuer` der Vollmachts-Issuer, `flows-pid-issuer` der PID-Issuer. Ein Release fährt genau einen Satz, weil die Sätze überlappende Routen beanspruchen.

Jeder Issuer liefert auch die Statusliste seiner eigenen Credentials aus (ADR-34): unter seinem Pfadpräfix als `GET /status-list/1`, signiert mit seinem eigenen Schlüssel als `statuslist+jwt` mit eingebetteter x5c-Kette, dazu ein `/admin`-Flow, über den ein Widerruf gesetzt wird. Die Widerrufsbits liegen auf dem Issuer-Volume und überleben einen Pod-Neustart. Einen separaten Status-Listen-Dienst brauchen die Issuer nicht.

Der PID-Issuer ist Dritter gegenüber **jeder** Instanz. Eine Instanz, die den Identitätsnachweis selbst ausstellt, den sie später als Beweis ihrer Unterzeichnenden akzeptiert, hat nichts bezeugt. Beim Vollmachts-Issuer liegt es umgekehrt: Er spricht *für* eine Organisation darüber, wer für sie zeichnen darf, und gehört deshalb zu ihr.

A7.1 Was der Betreiber beisteuern muss

Das Repository liefert die Form eines Issuers. Eine Identität liefert es nicht:

- **Eigene Hostnamen.** Jeder Host in den Beispielwerten ist ein Platzhalter, ebenso die Ingress-Klasse und die leere `tls`-Liste. Ein Issuer wird unter dem Hostnamen der Instanz veröffentlicht, der er dient, damit seine DID als `did:web:<host>:issuer` auflöst.
- **Eigene Schlüssel und DIDs.** Jeder Issuer erzeugt Root-CA und Signaturschlüssel beim Start in sein Volume und veröffentlicht sie unter seinem Pfadpräfix als `pki/root-ca.pem`, `pki/jwks.json` und `.well-known/did.json`.
- **Ein eigenes Vertrauensdokument je DCS-Instanz.** Nichts leitet es ab: es nennt Issuer über Bezeichner, die erst existieren, wenn die Issuer laufen.

A7.2 Reihenfolge und Befehle

Voraussetzungen: Namespaces existieren; Ingress-Klasse und TLS sind für die Ziel-Hostnamen geklärt; die DCS-Releases sind noch **nicht** installiert oder laufen ohne Vertrauensdokument.

Schritte:

1. Issuer neben jeder DCS-Instanz installieren:

```
helm install dcs-issuer ./deployment/helm/charts/orce \
  -n <namespace> -f ./deployment/helm/values.issuer.yml
```

2. PID-Issuer einmalig installieren:

```
helm install dcs-pid-issuer ./deployment/helm/charts/orce \
  -n <namespace> -f ./deployment/helm/values.pid-issuer.yml
```

3. Veröffentlichte Identitäten auslesen:

```
curl https://<host>/issuer/.well-known/did.json
curl https://<host>/issuer/pki/jwks.json
curl https://<host>/issuer/pki/root-ca.pem
curl https://<pid-host>/pid-issuer/pki/root-ca.pem
```

4. Vertrauensdokument je DCS-Instanz schreiben (Struktur siehe A7.3) und als ConfigMap anlegen:

```
kubectl create configmap dcs-oid4vp-trust -n <namespace> --from-file=trust.json=<datei>
```

5. Erst danach das `dcs`-Release installieren oder aktualisieren mit:

```
oid4vp:
  trust:
    enabled: true
    existingConfigMap: dcs-oid4vp-trust
    x5cAnchorsPath: /etc/dcs/oid4vp-x5c/root-ca.pem
```

An genau diesen Pfad wird über `volumes / volumeMounts` ein PEM-Bündel mit den Root-CAs **aller** Issuer gemountet, des Vollmachts- wie des PID-Issuers. Das Bündel wird aus dem zusammengesetzt, was jeder laufende Issuer unter `pki/root-ca.pem` tatsächlich ausliefert, und nach Fingerprint verglichen, nicht nach Subject: Die Demo-Issuer prägen ihre Roots beim ersten Start, alle tragen dasselbe Subject `CN = FACIS Demo Root CA`, und ein Bündel mit nur einem Root weist die Listen und Credentials der übrigen ab, während jede Fehlermeldung denselben CA-Namen nennt. Prägt ein Issuer seinen Root neu, etwa nach einem gelöschten Volume, ist das Bündel neu zu bauen; bis dahin schlägt die Prüfung fail-closed fehl.

Verifikation:

```
curl -o /dev/null -w '%{http_code}\n' https://<host>/issuer/.well-known/did.json
kubectl -n <ns> get configmap dcs-oid4vp-trust -o jsonpath='{.data.trust\.json}' | head -c 200
```

Der erste Aufruf muss `200` liefern, der zweite den Anfang des Dokuments. Das Backend-Log darf nach dem Rollout keinen Fehler beim Laden des Vertrauensdokuments zeigen.

Fehlerbilder:

Symptom	Ursache	Behebung
Render bricht mit Hinweis auf das Dev-Fixture ab	<code>oid4vp.trust.enabled=true</code> , aber kein <code>existingConfigMap</code> und kein <code>DCS_ALLOW_DEV_TRUST</code>	ConfigMap wie oben anlegen und referenzieren
Pod bleibt in <code>ContainerCreating</code> , Event nennt eine fehlende ConfigMap	<code>existingConfigMap</code> zeigt auf einen nicht existierenden Namen	ConfigMap anlegen. Der Mount ist bewusst nicht <code>optional</code> : Ein fehlendes Vertrauensdokument ist beim Start ohnehin fatal, und der Mount-Fehler lässt sich leichter diagnostizieren.
Backend beendet sich beim Laden des Vertrauensdokuments	Ein <code>mechanism</code> -Wert, den dieser Build nicht auflösen kann	Nur <code>jwtks</code> , <code>x5c</code> , <code>did:jwt</code> , <code>did:web</code> oder <code>orice</code> verwenden. Die Prüfung erfolgt beim Laden, nicht beim ersten Wallet-Kontakt.
Login funktioniert, Signaturzeremonie weist die Vollmacht ab	Dem eigenen Issuer fehlt der Zweck <code>peer</code>	Zweck ergänzen und das Backend neu ausrollen
Login oder Zeremonie scheitert an der Statuslisten- oder x5c-Prüfung, obwohl das Vertrauensdokument stimmt und die Liste korrekt signiert ist	Dem Anker-Bündel fehlt der Root eines Issuers. Alle Demo-Roots tragen dasselbe Subject; die Fehlermeldung nennt also einen CA-Namen, der im Bündel scheinbar vorhanden ist	Bündel aus den tatsächlich servierten Roots neu zusammensetzen und nach Fingerprint abgleichen (Schritt 5)

A7.3 Aufbau des Vertrauensdokuments

Vertrauen wird **je Zweck** gewährt (ADR-31):

Zweck	Bedeutung
<code>login</code>	Darf auf dieser Instanz eine Sitzung begründen
<code>peer</code>	Wird geprüft, wenn die Credential von einer Gegenpartei eintrifft, vor allem in der Signaturzeremonie
<code>pid</code>	Darf eine natürliche Person bezeugen

Jeder Issuer steht **zweimal** im Dokument: unter seinem `did:web`-Bezeichner, dessen Schlüssel live aus dem DID-Dokument des Issuers gelesen wird, und unter dem `https`-Bezeichner, den der Issuer tatsächlich in `iss` setzt. Der zweite ist nicht `did:web`-auflösbar und trägt seinen Schlüssel deshalb über den gewählten Mechanismus.

```
{
  "vcts": ["urn:dcs:poa:v1", "urn:dcs:pid:demo:v1"],
  "peer_dynamic": false,
  "issuers": {
    "did:web:dcs.example.org:issuer": {
      "purposes": ["login", "peer"],
      "organizations": ["did:web:dcs.example.org"],
      "mechanism": "did:web"
    },
    "https://dcs.example.org/issuer": {
      "purposes": ["login", "peer"],
      "organizations": ["did:web:dcs.example.org"],
      "mechanism": "jwtks",
      "jwtks": { "keys": [ "<der veröffentlichte JWKS des Issuers>" ] }
    },
    "did:web:pid.example.org:pid-issuer": { "purposes": ["pid"], "mechanism": "x5c" },
    "https://pid.example.org/pid-issuer": { "purposes": ["pid"], "mechanism": "x5c" }
  }
}
```

Regeln, die aus dem Modell folgen:

- **organizations** **begrenzt, worüber ein Issuer sprechen darf.** Eine Credential, die eine dort nicht gelistete Organisation nennt, wird auch bei gültiger Signatur abgewiesen. Der explizite Platzhalter `"*"` erlaubt jede Organisation und muss ausgeschrieben werden. Eine fehlende Liste bedeutet nicht „alle“.
- **Eine Instanz gewährt login und peer ihrem eigenen Issuer.** `peer` ist das, wogegen eine Signaturzeremonie die Vollmacht der lokalen Unterzeichnenden prüft. Es einem fremden Issuer zu gewähren, hieße, dass eine Gegenpartei sich selbst über ein von ihr veröffentlichtes Dokument autorisiert.
- **Der PID-Issuer erhält ausschließlich pid.** Ein Identitätsnachweis darf keinen Zugang gewähren.
- **Gegenparteien werden nicht aufgezählt.** Ob mit einer Gegenpartei überhaupt verkehrt wird, entscheidet der Trust-Gate über `DCS_TRUST_PDP_URL`, dynamisch und fail-closed.
- **Der Widerruf wird bei jeder akzeptierten Credential und jedem Zweck geprüft** und schlägt bei unerreichbarer Liste fehl. Eine unsignierte Statusliste wird nie akzeptiert (ADR-34). Eine Liste mit x5c-Kette wird gegen die Anker aus `x5cAnchorsPath` geprüft, eine ohne Kette gegen den im Vertrauensdokument hinterlegten JWKS ihres Issuers.

Das Dokument bindet Issuer-Schlüssel. Wird ein Issuer-Schlüssel neu erzeugt, verliert das Dokument stillschweigend seine Gültigkeit: Anmeldungen scheitern, ohne dass sich an der Konfiguration etwas geändert hätte. **Jede Neuerzeugung eines Issuer-Schlüssels erfordert eine Aktualisierung jedes Vertrauensdokuments, das ihn nennt.**

A7.4 Das Dev-Fixture

Bleibt `existingConfigMap` leer, fällt das Release auf ein im Image mitgeliefertes Fixture zurück, dessen Issuer-Schlüssel im Repository liegen. Wer eine Kopie des Repositories hat, kann damit Credentials prägen, die eine so konfigurierte Instanz akzeptiert. Das Chart verweigert deshalb den Render, es sei denn, das Release erklärt zusätzlich `DCS_ALLOW_DEV_TRUST=true` über `extraEnv`. Diese Erklärung gehört ausschließlich in einen Dev-oder CI-Stack.

Setzt ein Release `extraEnvFrom`, entfällt die Render-Zeit-Prüfung, weil deren Inhalt beim Rendern nicht lesbar ist; dann greift die inhaltsbasierte Prüfung im Backend selbst.

A8 Erst-Deployment

A8.1 Voraussetzungen

Alle folgenden Punkte müssen vor dem ersten `deploy.sh`-Aufruf erfüllt sein:

1. Ziel-Namespace existiert.
2. Images sind in einer für den Cluster erreichbaren Registry, Tags sind gepinnt, ein Pull-Secret ist vorhanden oder wird vom Chart erzeugt.
3. Die Credential-Issuer laufen und das Vertrauensdokument liegt als ConfigMap im Ziel-Namespace (A7). Reihenfolge: **Issuer vor DCS.**
4. Alle Secrets aus A6.4, die dieses Deployment referenziert, existieren.
5. Bei einem echten HSM: Token, alle fünf Schlüssel und das `did.json`-Secret existieren (A6.3).
6. Eine eigene Werte-Datei außerhalb dieses Repositories ist geschrieben. Sie trägt Hostnamen, DIDs, Registry, Secret-Referenzen und Storage-Klassen.
7. Der Release-Name steht fest. Er ist nicht kosmetisch: Subchart-Servicenamen leiten sich daraus ab, Werte wie `http://<release>-orice:1880` müssen dazu passen.
8. DNS zeigt für den öffentlichen Hostnamen auf den Cluster, TLS ist geklärt.

A8.2 Schritte

1. Werte und Templates gegen den Cluster prüfen, ohne etwas zu ändern:

```
./deployment/helm/deploy.sh --values <ihre-werte>.yaml \
                             --namespace <ns> --release <release> \
                             --dry-run
```

Erwartete Ausgabe: `Dry run passed: values and templates are valid and the API accepted every manifest.`

2. Deployment ausführen:

```
./deployment/helm/deploy.sh --values <ihre-werte>.yaml \
                             --namespace <ns> --release <release> \
                             --public-url https://<host>
```

`--values` ist wiederholbar; spätere Dateien überschreiben frühere. Die Chart-eigene `values.yaml` wird immer zuerst angewandt, es sind also nur die Abweichungen zu übergeben. Weitere Optionen: `--kubeconfig`, `--timeout` (Vorgabe `15m`), `--skip-verify`.

Das Skript baut zuerst die Abhängigkeiten aus dem `Chart.lock`, führt dann `helm upgrade --install` aus und prüft anschließend. Es ist konvergent: derselbe Aufruf gilt für Erstinstallation und Aktualisierung.

3. Das Skript wartet selbst auf den mitgelieferten Katalog (falls vorhanden) und auf den Rollout des Backends und führt danach die Prüfungen aus A8.3 aus.

Wichtig: **Kein** `helm --wait`. Helm hält `post-install`-Hooks zurück, bis alle Deployments bereit sind. Das Backend wird aber erst bereit, nachdem der HSM-Hook sein Material erzeugt hat. Das ist ein Deadlock. `deploy.sh` bricht ihn, indem es Helm ohne `--wait` aufruft und danach separat wartet.

A8.3 Verifikation

`deploy.sh` prüft nach dem Rollout Folgendes. Jede Prüfung entspricht einer Art, auf die eine Instanz gesund aussehen und trotzdem arbeitsunfähig sein kann; alle wurden auf einem laufenden Deployment beobachtet, das keine Fehler meldete.

Prüfung	Beobachtbares Signal	Was sie ausschließt
Backend-Start	<code>HTTP server listening</code> im Log	Ein Pod ist <code>Running</code> , bevor er seine Startgatter passiert hat, weil das Schlüsselmaterial erst nach dem Scheduling eintrifft
Katalog-Sync	Kein <code>failed to sync federated catalogue</code> im Log	Das Backend behandelt das als fatal und startet neu; sichtbar ist nur eine steigende Restart-Zahl
Katalog-Readiness	Kein <code>federated catalogue readiness gate failed</code> im Log	Ein 404 aus einem falsch gesetzten Kontextpfad
Trust-Gate	<code>DCS_TRUST_PDP_URL</code> gesetzt und der Endpunkt antwortet 2xx	Der Gate ist fail-closed. Unerreichbar heißt: Föderation vollständig aus, ohne dass die Instanz etwas meldet. Eine erreichbare URL genügt nicht, der Endpunkt muss antworten.
Validator	<code>DSS_URL</code> gesetzt	Ohne Validator weist die Instanz jede extern erzeugte Signatur ab. Vertragserstellung funktioniert weiter, der Ausfall zeigt sich erst beim Signieren.
Storage-Klassen	Postgres-, IPFS- und HSM-Token-PVC an die konfigurierte Klasse gebunden	Ein bestehender Claim behält seine Klasse; eine spätere Werte-Änderung bleibt wirkungslos
Föderationsdokumente	<code>/well-known/did.json</code> , <code>/well-known/dcs-agreement-credential.json</code> und <code>/well-known/dcs-federation-rules.md</code> liefern von außen 200	Hydras OIDC-Discovery erhebt einen zweiten Anspruch auf <code>/well-known</code> und überdeckt sie. Im Cluster sieht alles korrekt aus; nur ein externer Abruf zeigt den 404.
DID-Übereinstimmung	Das ausgelieferte <code>did.json</code> trägt dieselbe <code>id</code> wie <code>ISSUER_DID</code>	Eine Abweichung lässt Peers die Instanz ablehnen, sichtbar nur auf der Gegenseite

Die externen Prüfungen laufen nur mit `--public-url`. Ohne sie sagt das Skript das ausdrücklich, statt still durchzulaufen.

Zwei weitere Signale, die sich lohnen und die das Skript nicht abdeckt:

```
# Alle Pods laufen oder sind abgeschlossen
kubectl get pods -n <ns> --no-headers | grep -vE 'Running|Completed'

# Der Login-Einstieg antwortet
curl -o /dev/null -w '%{http_code}\n' -X POST https://<host><basePath>/api/auth/login
```

Die erste Zeile darf nichts ausgeben, die zweite `200`.

Zur Ausgabe, die Helm nach dem Install anzeigt: Bei aktiviertem Ingress nennt sie die korrekte URL einschließlich `route.basePath`. Der ClusterIP-Zweig dagegen schlägt einen Port-Forward vor und nennt anschließend `http://127.0.0.1:8991` **ohne** den Basispfad. Wer dieser Adresse folgt, landet auf nichts. Die vollständige lokale Adresse ist `http://127.0.0.1:<lokaler-port><basePath>/ui/`.

A8.4 Fehlerbilder beim Erst-Deployment

Siehe A13.

A9 GitOps über ArgoCD

Dieser Weg ist nie getestet worden. Weder in der CI noch in einer Abnahmeumgebung ist je ein DCS-Release über ArgoCD ausgerollt worden. Das Manifest unten ist aus den Artefakten abgeleitet und plausibel, aber unerprobt. Wer ihn geht, ist der Erste. Plane entsprechend: Probier ihn zuerst gegen einen Wegwerf-Namespace, nicht gegen die Abnahmeumgebung, und rechne damit, dass die Reihenfolge der Provisionierungs-Hooks (A9.1) der Punkt ist, an dem es klemmt. Der erprobte Weg für Erstinstallation und Upgrade ist `deploy.sh` (A8, A11).

Das Chart bringt ein Application-Manifest mit: `deployment/argocd/application.yaml`.

```
kubectl apply -n argocd -f deployment/argocd/application.yaml
```

Das mitgelieferte Manifest beschreibt die Abnahmeumgebung (`dcS-acceptance`, Namespace `dcS-acceptance`, `valueFiles: values.yaml + values.acceptance.yaml`). Für Produktion wird es dupliziert und `valueFiles` sowie `destination` angepasst.

Drei Stellen im mitgelieferten Manifest sind vor dem ersten Anwenden zu ersetzen:

- `repoURL` trägt eine SSH-URL auf ein konkretes Fork-Repository. Sie ist auf das eigene Repository zu setzen, und ArgoCD braucht dafür hinterlegte Zugangsdaten.
- `targetRevision` steht auf `main`. Für einen Betrieb gehört dort ein fester Stand, nicht ein beweglicher Branch.
- `valueFiles` verweist auf `values.acceptance.yaml`, ein Skelett. Diese Datei allein rendert nicht, weil `systemToken` in ihr fehlt (A4.1). Ein ArgoCD-Sync gegen das unveränderte Manifest schlägt deshalb schon beim Rendern fehl. Die eigene Werte-Datei muss ergänzt werden, und die Secrets daraus dürfen nicht im GitOps-Repository liegen.

Gesetzte `syncOptions: CreateNamespace=true` und `ServerSideApply=true`. Die Retry-Politik ist fünf Versuche mit exponentiellem Backoff von 5s bis 3m.

A9.1 Warum Auto-Sync bewusst aus ist

`syncPolicy.automated` ist nicht gesetzt. Dahinter stehen vier Gründe, drei aus dem Chart und einer aus dem Betrieb:

1. **Ein unbeaufsichtigtes `selfHeal` kann eine gewollte Handlung als Drift maskieren.** Eine Schlüsselrotationsübung sieht für einen Reconciler wie eine Abweichung vom gerenderten Zustand aus.
2. **Der FC-Realm-Hook muss laufen, bevor die Anwendung gesund werden kann.** Die Realm-Provisionierung ist ein `post-install,post-upgrade` -Hook. Ein Reconciler, der Helm-Hooks auf eigene Phasen abbildet, führt `post-*` typischerweise erst aus, wenn der Sync **gesund** ist. Das Backend behandelt einen fehlgeschlagenen Katalog-Sync aber als fatal und läuft in eine Neustartschleife, bis der Hook seinem Dienstkonto die Rollen zuteilt. Der Hook, der die Anwendung entsperrt, läuft nie, weil die Anwendung nie gesund wird. Plain `helm` koppelt `post-install` -Hooks nicht an die Pod-Gesundheit, dort löst sich die Reihenfolge von selbst.
3. **Komponenten des Charts patchen, was das Chart rendert.** Der Realm-Hook schreibt die FC-Issuer-URL in `fc-service`, nachdem die Manifeste angewandt wurden. Ein Reconciler, der Live-Zustand gegen gerenderten Zustand vergleicht, sieht dort dauerhafte Drift und arbeitet dagegen an.
4. **Provisionierung ist ein First-Boot-Seiteneffekt, kein deklarativer Zustand.** Schlüsselmaterial, `did.json` und die C2PA-x5chain entstehen einmalig in ein Volume. Ein Reconciler kann nicht wissen, dass ein erneuter Provisionierungslauf nur deshalb ungefährlich ist, weil die Skripte idempotent sind.

Der empfohlene Ablauf: ArgoCD hält den Sollzustand, meldet Drift und erzeugt den Sync-Auftrag. Ausgelöst wird der Sync von Hand oder über eine Freigabe in der Pipeline. Für Erstinstallation und für Upgrades, die die Provisionierungs-Hooks berühren, ist `deploy.sh` der verlässlichere Weg, weil es die Reihenfolge kennt.

`automated.prune` / `selfHeal` können eingeschaltet werden, sobald der Betreiber die vier Punkte für seine Umgebung geprüft hat.

Verifikation eines ArgoCD-Deployments:

```
kubectl -n argocd get application dcS-acceptance -o jsonpath='{.status.sync.status} {.status.health.status}{"\n"}'
```

Erwartet: `Synced Healthy`. Danach gelten dieselben inhaltlichen Prüfungen wie in A8.3, vor allem die externen `/.well-known` -Abrufe, die ArgoCD nicht kennt.

A10 CI/CD über GitHub Actions

Workflow	Auslöser	Wirkung
<code>publish.yml</code>	Veröffentlichter Release, manuell	Baut das DCS-Image und das pdf-core-Image über den <code>eclipse-xfsc/dev-ops</code> -Dockerbuild-Workflow nach Harbor; packt das Chart und pusht es als OCI-Artefakt nach <code>oci://<HARBOR_HOST>/dcs/charts</code>
<code>bdd-kind.yml</code>	Pull Request, Push auf <code>main</code> , manuell	Drei parallele Jobs: die vollständige BDD-Suite auf einem eigenen kind-Cluster, ein isolierter Audit-Gate, und die Playwright-E2E-Suite auf einem weiteren kind-Cluster. Berichte werden als Check-Annotationen und Artefakte veröffentlicht.
<code>lint-and-format.yml</code>	siehe Workflow-Datei	Statische Prüfungen
<code>build-node-red-package.yml</code>	Veröffentlichter Release, Pull Request, Push auf <code>main</code> , manuell	Baut das Node-RED-Deployment-Paket aus <code>deployment/node-red</code> ; bei einem Release wird das <code>.tgz</code> an diesen angehängt
<code>sbom.yml</code>	Veröffentlichter Release, monatlich, manuell	Ruft den zentralen SBOM-Workflow auf
<code>eclipse-dash.yml</code>	Veröffentlichter Release, monatlich, manuell	Lizenz-Scan über Eclipse Dash

Die Chart-Veröffentlichung baut die Abhängigkeiten explizit neu, weil `charts/*.tgz` nicht im Repository liegt: die lokalen `file://` -Subcharts lösen aus ihren Quellverzeichnissen auf, die externen Abhängigkeiten (`kube-prometheus-stack`, `traefik`) müssen geholt werden.

Verifikation einer Veröffentlichung:

```
helm show chart oci://<HARBOR_HOST>/dcs/charts/digital-contracting-service --version <version>
```

Muss die erwartete `version` und `appVersion` ausgeben.

A11 Upgrade, Migration und Rollback

A11.1 Was bei einem Upgrade automatisch passiert

Vorgang	Ausgelöst durch	Anmerkung
Datenbank-Migrationen	Backend beim Start	Alle noch nicht angewandten SQL-Dateien werden alphabetisch sortiert angewandt und in <code>schema_migrations</code> vermerkt. Es gibt keine Rückwärtsmigrationen.
Semantic-Hub-Seed und Anker-Refresh	Backend beim Start	Bricht der Vorgang ab, startet die Instanz nicht
HSM-Provisionierung	<code>post-install</code> , <code>post-upgrade</code> -Hook	Idempotent: vorhandener Token und vorhandene Schlüssel bleiben unangetastet
FC-REALM-Provisionierung	<code>post-install</code> , <code>post-upgrade</code> -Hook	Idempotent; das Backend wartet über Init-Container auf Erstellung und Abschluss des Jobs
Seeding der Zielsystem-Registry	Backend beim Start	Nur anlegend, nie ändernd oder löschend
Seeding der Maschinen-Identitäten	Backend beim Start	Aus <code>DCS_SYSTEM_CLIENTS</code>

A11.2 Upgrade-Prozedur

Voraussetzungen:

- Eine aktuelle Sicherung nach `docs/backup-integration-guide.md` liegt vor, vor allem die der `dcs` -Datenbank, weil die gewrappten CEKs nur dort existieren.
- Die Release-Notes des Zielstands sind auf neue Pflichtkonfiguration geprüft. Eine neue Pflichtvariable ist der häufigste Grund, warum ein Upgrade ein Deployment stilllegt, das vorher lief.
- Bei einem echten HSM: Wenn der neue Stand einen zusätzlichen Schlüssel verlangt, existiert er bereits im Token und ist im `did.json` veröffentlicht. Das Chart legt in Produktion keine HSM-Schlüssel an.
- Der Ziel-Image-Tag ist gepinnt.

Schritte:

- Trockenlauf gegen den Cluster:

```
./deployment/helm/deploy.sh --values <ihre-werte>.yaml \
--namespace <ns> --release <release> --dry-run
```

2. Aktuelle Werte des laufenden Release sichern. Bei manuell nachgezogener Konfiguration ist das die einzige Aufzeichnung:

```
helm -n <ns> get values <release> > <release>-werte-vor-upgrade.yaml
```

3. Upgrade ausführen (derselbe Aufruf wie beim Erst-Deployment):

```
./deployment/helm/deploy.sh --values <ihre-werte>.yaml \
--namespace <ns> --release <release> \
--public-url https://<host>
```

Verifikation: Die vollständige Prüfliste aus A8.3. Zusätzlich:

```
helm -n <ns> history <release>
kubectl -n <ns> get pods -l app.kubernetes.io/instance=<release> \
-o 'custom-columns=POD:.metadata.name,READY:.status.containerStatuses[*].ready,RESTARTS:.status.containerStatuses[*].restartCount'
```

Die neue Revision muss `deployed` sein, und die Restart-Zahl des Backend-Containers darf nach dem Rollout nicht weiter steigen. Eine steigende Restart-Zahl bei `Running`-Status ist das Bild eines fatalen Startfehlers.

Fehlerbilder: siehe A13.

A11.3 Rollback und seine Grenzen

```
helm -n <ns> rollback <release> <revision>
```

Was ein Rollback **nicht** rückgängig macht, und was daraus folgt:

- **Schema-Migrationen.** Sie sind vorwärtsgerichtet und haben keine Gegenstücke. Ein Rollback auf ein älteres Image gegen eine bereits migrierte Datenbank ist nur zulässig, wenn der ältere Stand das neuere Schema toleriert. Ist er das nicht, ist der Weg die Wiederherstellung nach `docs/backup-integration-guide.md`, nicht ein Helm-Rollback.
- **Provisioniertes Schlüsselmaterial.** Token, `did.json` und `x5chain` liegen auf einem Volume und bleiben, wie sie sind. Das ist erwünscht: ein Rollback darf die Identität der Instanz nicht ändern.
- **Von Hooks geschriebene Zustände:** der Keycloak-Realm, die Rollenzuordnung des Katalog-Dienstkontos.
- **Registry-Einträge,** die über die Oberfläche geändert wurden. Das `contractTargets`-Seeding legt nur an.

Ein Rollback nimmt also Konfiguration und Images zurück, kein Datenmodell. Diese Prozedur ist nicht in einem laufenden Cluster erprobt worden, sie ist aus den Artefakten abgeleitet.

A11.4 Schlüsselrotation

Signatur Schlüssel sind versioniert: Version 1 ist das unfixierte Basislabel, jede Rotation legt einen `-v<N>`-Schlüssel daneben. Alte Versionen bleiben im Token, damit historische Signaturen prüfbar bleiben.

```
DATABASE_URL=<dsn> bash scripts/rotate-hsm-key.sh \
<token-dir> <token-label> <pin> <basis-label> [modul] [ca-dir] [x5chain-out]
```

Das Skript erzeugt das nächste Schlüsselpaar (bei `dcS-ecdh` mit `--usage-derive`), stellt optional ein Blattzertifikat gegen die Dev-CA aus und rückt den Zeiger in `pki_active_key_version` vor. `DATABASE_URL` ist Pflicht; ohne sie bricht das Skript ab.

Es ist auf ein SoftHSM2-Token zugeschnitten. Gegen ein Hersteller-HSM ist die Schlüsselerzeugung durch die Herstellerprozedur zu ersetzen; der Zeiger-Vorschub in `pki_active_key_version` bleibt derselbe Schritt.

Rotiert man `dcS-ecdh`, muss das veröffentlichte `did.json` den neuen `keyAgreement`-Eintrag tragen, sonst schlägt der Selbsttest beim nächsten Start fehl. Rotiert man einen Issuer-Schlüssel, ist jedes Vertrauensdokument anzupassen, das ihn nennt (A7.3).

A12 Betriebsverhalten und Verifikation

A12.1 Was der Betreiber sieht

Readiness. Das Backend beansprucht seine Listen-Adresse sofort und antwortet auf `/readyz` mit 503, bis die Initialisierung abgeschlossen ist. Dazu gehören das Öffnen des PKCS#11-Tokens, das funktionale Katalog-Gate und der Schema-Sync. Die Readiness-Probe fragt `/readyz` alle 2 Sekunden mit 1 Sekunde Timeout und 3 Fehlversuchen ab.

Ein Pod im Zustand `Running` mit `0/1 Ready` und einem Log, das `bootstrap HTTP server listening ... (initializing)` zeigt, ist in den ersten Minuten nach einem Install **normal**. Das muss kein Fehler sein.

Liveness. Der Backend-Container hat per Vorgabe keine Liveness-Probe (A5.7). pdf-core hat beides: Readiness und Liveness auf `/swagger.json`.

Metriken. Das Backend exponiert Prometheus-Metriken unter `/metrics` auf der Wurzel des Service-Ports, also außerhalb von `DCS_API_PATH`. Das Scrape-Ziel dafür bringt der Betreiber mit; ein für den Betrieb verwendbares ServiceMonitor-Objekt liefert das Chart nicht.

Log-Signale, an denen ein Betreiber den Zustand ablesen kann:

Zeile	Bedeutung
<code>bootstrap HTTP server listening on ... (initializing)</code>	Prozess läuft, Initialisierung noch nicht durch
<code>waiting for PKCS#11 token (attempt N): ...</code>	Das Token ist noch nicht da. Wird sofort und danach etwa einmal pro Minute geloggt; der Versuch wird alle 5 Sekunden wiederholt. Der Prozess stirbt daran nicht .
<code>HTTP server listening on ...</code>	Initialisierung abgeschlossen
<code>failed to sync federated catalogue schemas</code>	Fatal; die Instanz startet neu. <code>deploy.sh</code> sucht nach dem Präfix <code>failed to sync federated catalogue</code>
<code>incomplete Federated Catalogue configuration; missing ...</code>	Fatal; die Meldung nennt die fehlenden Variablen namentlich
<code>federated catalogue readiness gate failed: ...</code>	Fatal; typischerweise ein falscher Katalog-Kontextpfad
<code>Could not read did document</code>	<code>DCS_DID</code> fehlt oder zeigt ins Leere
<code>did.json carries no usable keyAgreement method for dcs-ecdh</code>	Das veröffentlichte DID-Dokument und der HSM-Schlüsselsatz passen nicht zusammen

Geplante Jobs.

Job	Vorgabe-Takt	Zweck
<code><fullname>-pac-monitor</code>	alle 5 Minuten, <code>concurrencyPolicy: Forbid</code> , <code>startingDeadlineSeconds: 120</code>	Compliance-Durchlauf. Ein verpasster Lauf wird nicht nachgeholt; der nächste leitet denselben Zustand neu ab. Historie: 1 Erfolg, 3 Fehlschläge.
<code><fullname>-acme</code>	<code>17 3 * * 1,4</code>	Nur mit <code>acmeRenewal.enabled</code> . Zweimal wöchentlich; certbot erneuert nur innerhalb seines Fensters und beendet sich sonst mit 0

Ein fehlgeschlagener Compliance-Durchlauf ist ein sichtbares Kubernetes-Objekt, keine Logzeile:

```
kubectl -n <ns> get jobs -l app.kubernetes.io/name=<release>-digital-contracting-service-pac-monitor
```

Wiederholungen. Das Backend wiederholt fehlgeschlagene PDF-Auslieferungen an Peers und fehlgeschlagene PDF-Regenerationen in einem eigenen Takt. Die Produktionsvorgabe liegt im Minutenbereich; die Test-Werte setzen ihn über `DCS_SYNC_FAIL_RETRY_INTERVAL` und `DCS_PDF_REGENERATION_RETRY_INTERVAL` auf 10s herunter.

Startgatter. Diese Konfigurationen sind beim Start zwingend; fehlt eine, läuft die Instanz nicht an: `DCS_PUBLIC_URL`, `DCS_PUBLIC_BASE_URL`, `DCS_DID`, `TSA_URL`, `STATUSLIST_SERVICE_URL` (und der Dienst muss antworten), `PDF_CORE_URL`, `IPFS_TENANT_BASE_URL` und `IPFS_MFS_BASE_URL`, ein vollständiger Federated-Catalogue-Block (sobald einer seiner Werte gesetzt ist), `DCS_SYSTEM_TOKEN` (bereits beim Render), sowie ein ladbares Vertrauensdokument.

A12.2 Wiederkehrende Betriebsprüfung

```
# 1. Nichts hängt
kubectl get pods -n <ns> --no-headers | grep -vE 'Running|Completed'

# 2. Die Instanz ist von außen auflösbar
for p in $(cat /dev/null); do
  printf '%s ' "$p"; curl -s -o /dev/null -w '%{http_code}\n' "https://<host>$p"
done

# 3. Der Trust-Gate antwortet
kubectl -n <ns> exec deployment/<release>-digital-contracting-service -- \
  sh -c "curl -s -o /dev/null -w '%{http_code}' -X POST -H 'Content-Type: application/json' \
  -d '{\"peerDID\": \"did:web:probe\", \"direction\": \"inbound\"}' \"\${DCS_TRUST_PDP_URL}\""

# 4. Die Volumes tragen noch die vorgesehene Storage-Klasse
kubectl -n <ns> get pvc -o 'custom-columns=NAME:.metadata.name,CLASS:.spec.storageClassName,PHASE:.status.phase'
```

Erwartet: (1) keine Ausgabe, (2) dreimal `200`, (3) ein `2xx`-Code, (4) alle Claims `Bound` mit der konfigurierten Klasse.

A13 Fehlerbilder

Alle folgenden Fälle sind auf laufenden Deployments beobachtet worden oder folgen zwingend aus den Artefakten. Das Symptom ist jeweils das, was der Betreiber sieht.

A13.1 Installation und Start

Symptom	Ursache	Behebung
helm bricht ab mit <code>systemToken must be set: ...</code>	Die verwendete Werte-Datei deklariert <code>systemToken</code> nicht (betrifft <code>values.prod.yml</code> und <code>values.acceptance.yml</code>)	Einen instanzeigenen Zufallswert über <code>--set</code> aus dem Secret-Manager übergeben (A6.6)
Render bricht ab mit dem Hinweis auf das Dev-Trust-Fixture	<code>oid4vp.trust</code> zeigt auf das im Image liegende Fixture, ohne dass das Release <code>DCS_ALLOW_DEV_TRUST=true</code> erklärt	Vertrauensdokument als ConfigMap anlegen und referenzieren (A7)
Render bricht ab mit <code>remote Federated Catalogue requires federatedCatalogue.remote.acknowledgeAdminAllTrustBoundary=true</code>	<code>federatedCatalogue.enabled</code> ohne <code>fcservice.enabled</code>	Die Vertrauensgrenze bewusst bestätigen oder einen eigenen Katalog betreiben (A5.5)
Render bricht ab mit <code>ingress.enabled=true requires at least one entry in ingress.hosts</code>	<code>ingress.hosts</code> ist leer	Hostnamen eintragen
Render bricht ab mit <code>pdfCore.signing.signerKeyPEM is required ...</code>	<code>pkcs11.provisioning.enabled=false</code> und <code>pdfCore.signing.existingSecret</code> leer	<code>pdfCore.signing.existingSecret</code> auf das vorab angelegte Secret mit der C2PA-Kette setzen (A6.3). Das ist der Weg für Produktion.
Backend-Pod Running, aber dauerhaft 0/1 Ready; Log wiederholt <code>waiting for PKCS#11 token (attempt N): CKR_GENERAL_ERROR</code>	Der HSM-Provisionierungs-Job hat nie einen Pod gestartet. Tritt insbesondere auf, wenn Manifeste ohne Helm angewandt wurden: ein bestehender Job wird gepatcht statt neu ausgeführt	Job löschen und neu anlegen. Das Backend pollt, statt abzustürzen, und erholt sich ohne Neustart
Backend-Log <code>federated catalogue readiness gate failed: health check failed with status 404</code>	<code>federatedCatalogue.apiUrl</code> ohne den Servlet-Kontextpfad des fc-service. <code>fc-service</code> läuft mit <code>SERVER_SERVLET_CONTEXT_PATH=/fc</code> ; eine In-Cluster-URL ohne <code>/fc</code> liefert auf jeden Aufruf 404	Kontextpfad an die URL anhängen. Achtung auf die Asymmetrie: eine öffentliche URL wie <code>https://<host>/fc</code> enthält ihn bereits, eine In-Cluster-URL <code>http://fc-service:8081</code> nicht
Backend beendet sich mit <code>incomplete Federated Catalogue configuration; missing ...</code>	Der Katalog-Block ist alles-oder-nichts: sobald eine der vier Variablen gesetzt ist, müssen alle vier gesetzt sein	Die in der Meldung genannten Werte ergänzen: <code>FEDERATED_CATALOGUE_API_URL</code> , <code>FEDERATED_CATALOGUE_CLIENT_ID</code> , <code>FEDERATED_CATALOGUE_CLIENT_SECRET</code> , <code>FC_KEYCLOAK_REALM_URL</code>
Backend beendet sich mit <code>Could not read did document</code>	<code>identity.enabled</code> ist nicht gesetzt, also wird <code>DCS_DID</code> nicht gerendert	<code>identity.enabled: true</code> setzen und sicherstellen, dass die Identitätsquelle existiert
Backend beendet sich mit <code>did document has no keyAgreement method with suffix "#dcs-ecdh"</code>	Das veröffentlichte <code>did.json</code> stammt aus einer Quelle ohne <code>keyAgreement</code> -Eintrag, oder dem Token fehlt der <code>dcs-ecdh</code> -Schlüssel mit <code>CKA_DERIVE</code>	Bei SoftHSM2: den Provisionierungs-Job neu laufen lassen. Bei echtem HSM: Schlüssel mit Derive-Fähigkeit anlegen und <code>did.json</code> neu erzeugen
pdf-core-Pod bleibt im Init-Container <code>wait-for-x5chain</code>	Das x5chain-Secret bzw. die Datei auf dem Token-Volume existiert noch nicht	Provisionierungs-Job prüfen. Im Produktionsmodus: das Secret aus A6.3 anlegen
helm install schlägt fehl mit <code>cannot create resource roles</code>	Die installierende Identität darf keine RBAC-Objekte anlegen	Betrieb ohne RBAC-Rechte nach A6.5

A13.2 Ressourcen und Scheduling

Symptom	Ursache	Behebung
Ein Job steht auf <code>Running</code> mit null Pods; Event: <code>forbidden: minimum cpu usage per Container is 100m, but request is 50m</code>	Eine <code>LimitRange</code> weist ab , sie klemmt nicht hoch. Der Job-Controller versucht es endlos weiter, was wie ein Hänger aussieht statt wie eine Ablehnung	Die tatsächlichen Werte der <code>LimitRange</code> lesen (<code>kubectl -n <ns> describe limitrange</code>) und die <code>Requests</code> darüber setzen
Deployments existieren mit null Replikaten, aber kein Install-Fehler	<code>limits.cpu</code> der <code>ResourceQuota</code> ist erschöpft. Ein Container ohne <code>resources</code> - Block erbt den <code>LimitRange</code> -Default (oft 1 CPU) und verbraucht die Quota	Quota erhöhen, oder explizite kleine <code>resources.limits</code> setzen. Zu dimensionieren sind: <code>resources</code> , <code>pdfCore.resources</code> , <code>pkcs11.provisioning.resources</code> , <code>fcservice.realmProvision.resources</code> , <code>fcservice.resources</code> , sowie <code>resources / jobResources</code> der <code>Issuer-Releases</code>
Pods bleiben <code>Pending</code> , PVC ist <code>Pending</code>	Keine <code>Default-StorageClass</code> , oder die konfigurierte Klasse existiert nicht, oder die angeforderte Größe liegt unter dem Minimum des CSI-Treibers	Klasse setzen. Der HSM-Token fordert bewusst 1 Gi statt der tatsächlich benötigten ~10 MiB, weil mehrere Treiber darunter nicht provisionieren
TLS wird nie ausgestellt, keine Fehlermeldung	Eine <code>services</code> -Obergrenze der Quota lässt den ACME-HTTP-01-Solver von <code>cert-manager</code> nicht mehr zu; er braucht einen eigenen Service. Alternativ verhindert ein <code>LimitRange</code> -Minimum, dass der Solver-Pod <code>scheduled</code>	Quota erhöhen, oder den <code>CronJob</code> -Fallback <code>acmeRenewal</code> verwenden (A5.6)

A13.3 Netzwerk, Ingress und Föderation

Symptom	Ursache	Behebung
<code>https://<host>/ .well-known/did.json</code> liefert 404, im Cluster sieht alles korrekt aus	Hydras <code>OIDC-Discovery</code> beansprucht per Vorgabe den gesamten <code>/ .well-known</code> -Präfix und überdeckt die drei DCS-Dokumente	Die drei Pfade als exakte Regeln vor der Hydra-Regel routen. Unter <code>ingress-nginx</code> tut das <code>Chart</code> das von selbst; unter anderen Controllern sind sie wie in <code>values.bdd.yml</code> als vollständige Pfade zu listen
Ein Peer weist die Instanz ab, lokal ist nichts auffällig	Das ausgelieferte <code>did.json</code> trägt eine andere <code>id</code> als <code>ISSUER_DID</code>	Beide angleichen und das Backend neu ausrollen. <code>deploy.sh -public-url</code> prüft das
Föderation transportiert nichts, keinerlei Fehlermeldung	Der Trust-Gate ist fail-closed. <code>DCS_TRUST_PDP_URL</code> ist nicht gesetzt oder der Endpunkt antwortet nicht mit 2xx	<code>federation.trustPdpURL</code> setzen und die Erreichbarkeit prüfen (A12.2). Eine erreichbare URL genügt nicht
Zwei Instanzen föderieren nicht, obwohl beide erreichbar sind	Beide müssen <code>/ .well-known/dcs-agreement-credential.json</code> veröffentlichen, und das Agreement vergleicht einen Hash der eingebetteten Föderationsregeln. Zwei Instanzen föderieren nur mit demselben Build	Beide Seiten auf denselben Stand bringen
Token-Anforderung beim Katalog-Keycloak liefert 401	<code>federatedCatalogue.oauth.clientID</code> benennt einen Client, den der Realm nicht anlegt. Die Chart-Vorgabe <code>dcs-fc-client</code> existiert im mitgelieferten Realm nicht	Gegen den mitgelieferten Katalog <code>federated-catalogue</code> setzen; gegen einen externen Keycloak den dort angelegten Client
Katalog-Aufrufe scheitern durchgängig mit 401	Die Realm-URL, gegen die <code>fc-service</code> den <code>iss</code> -Claim prüft, ist eine In-Cluster-Adresse, das Token wurde aber über die öffentliche geprägt (oder umgekehrt). Beide Seiten müssen sich auf dieselbe Adresse einigen	Die öffentliche Realm-URL in <code>fcKeycloak.realmURL</code> setzen und <code>fcservice.keycloak.hostname</code> mit <code>fcKeycloakApp.hostname</code> synchron halten. Das ist Handarbeit, weil eine Werte-Datei sich nicht selbst referenzieren kann
<code>/ .well-known/openid-configuration</code> des Katalog-Keycloak liefert 200, die Token-Ausstellung aber 500	Die Katalog-Datenbank wurde zurückgesetzt, Keycloak lief weiter und lieferte Realm-Metadaten aus dem Speicher über einem nicht mehr existierenden Schema	<code>fcservice.realmProvision.enabled=true</code> setzen und den Realm neu importieren. Vorbeugend: <code>fcPostgres.persistence.enabled</code> und <code>fcFuseki.persistence.enabled</code> einschalten, denn die Vorgabe ist <code>emptyDir</code>
Ein zweites, altes DCS-Release auf demselben Host lässt <code><pfad>/ .well-known/did.json</code> mit 200 antworten, während die Wurzel 404 liefert	Zwei Releases auf einem Hostnamen. Der Abruf des falschen Pfades liest sich wie „Instanz läuft“	Ein Host, eine DCS-Instanz. Altrelease samt Ingress, Jobs, Release-Secrets und PVCs entfernen

A13.4 Werkzeug- und Ablaufprobleme

Symptom	Ursache	Behebung
Chart-Änderungen an einem Subchart wirken nicht	Helm bevorzugt ein gepacktes <code>charts/*.tgz</code> gegenüber dem Quellverzeichnis; ein altes Archiv liefert alte Templates aus	<code>helm dependency build --skip-refresh deployment/helm</code> vor jedem Rendern. <code>deploy.sh</code> tut das von sich aus
<code>helm list</code> zeigt nichts, obwohl Objekte im Namespace liegen	Der Install wurde abgebrochen und steht auf <code>pending-install</code> . Für <code>helm list</code> ist das unsichtbar, für <code>helm status</code> nicht	<code>helm -n <ns> status <release></code> prüfen; danach gezielt aufräumen oder mit <code>helm upgrade --install</code> adoptieren
Ein Deployment erzeugt gar nichts, ohne Fehler	Zwei gleichzeitige Operationen auf demselben Release. Die Sperre gilt pro Release-Name; verschiedene Releases sind unkritisch	Vor dem Start <code>pgrep -af helm</code> prüfen
Ein abgebrochenes Deployment hinterlässt einen unaufgeräumten Release	Ein <code>helm</code> -Aufruf wurde von außen abgebrochen, etwa durch ein Timeout des aufrufenden Werkzeugs. Der Kindprozess überlebt einen getöteten Shell und läuft dem Aufräumen in die Quere	Deployment nie in einen <code>timeout</code> wickeln und nie in einem Werkzeugaufruf mit kürzerer Frist als Helms eigenem <code>--timeout</code> starten
<code>helm install</code> schreibt über eine langsame oder instabile Verbindung etwa ein Objekt alle 20 Minuten und meldet nie einen Fehler	Der langlebige Helm-Client verträgt die Verbindung nicht und wiederholt mit langem Backoff. Erkennbar daran, dass <code>helm status</code> , <code>Discovery</code> und <code>helm install --dry-run=server</code> sofort antworten und <code>kubectl apply</code> derselben Manifeste in Sekunden durchläuft	Notweg in A13.5. Zuerst die Verbindung prüfen, denn dieser Weg hat einen Preis
Ein <code>kubectl apply</code> auf ein bestehendes Deployment scheitert mit <code>may not specify more than 1 handler type</code>	Server-side Apply mischt Probes. Eine alte <code>exec</code> -Probe steht dann neben der <code>httpGet</code> -Probe des Charts	Alte entfernen und neue setzen in einem JSON-Patch, oder das Deployment löschen und neu anlegen
Ein PVC bleibt in <code>Terminating</code>	<code>pvc-protection</code> hält den Claim, solange irgendein Pod ihn referenziert, auch ein <code>Succeeded</code> -Job-Pod	Die Job-Pods löschen, dann das Volume

A13.5 Notweg: rendern mit Helm, anwenden mit kubectl

Nur zu verwenden, wenn der Helm-Client die Verbindung zum Cluster nicht verträgt (A13.4) und das Deployment nicht warten kann.

```
helm template <release> deployment/helm -n <ns> -f <ihre-werte>.yaml > /tmp/render.yaml
```

Die gerenderten Objekte werden anschließend als gewöhnliche Objekte angewandt. Damit ein späteres `helm upgrade --install` sie adoptiert statt mit ihnen zu kollidieren, müssen sie die Helm-Besitzmarkierungen tragen: das Label `app.kubernetes.io/managed-by: Helm` sowie die Annotationen `meta.helm.sh/release-name` und `meta.helm.sh/release-namespace`. `helm.sh/hook`-Annotationen werden dabei entfernt (sonst behandelt Helm die Objekte später als Hooks), und `helm.sh/hook: test`-Objekte werden ausgelassen.

```
kubectl apply -n <ns> -f /tmp/objekte.yaml
```

Der Preis, und den muss man kennen:

- **Es gibt keinen Helm-Release-Datensatz.** `helm list` zeigt nichts, `helm history` und `helm rollback` existieren für dieses Deployment nicht. Sobald die Verbindung wieder trägt, ist mit `helm upgrade --install <release> deployment/helm -n <ns> -f <werte>.yaml` zu adoptieren. Möglich wird das durch die Besitzmarkierungen oben.
- **Helms `before-hook-creation`-Löschregel entfällt.** Provisionierungs-Jobs werden gepatcht, nicht neu ausgeführt. Bei einer frischen Installation bedeutet das: der HSM-Job läuft nie, das Backend wartet endlos auf sein Token. Die betroffenen Jobs sind vor dem Anwenden zu löschen:

```
kubectl delete job -n <ns> <release>-digital-contracting-service-hsm-provision
kubectl delete job -n <ns> <release>-digital-contracting-service-fc-realm-provision
```

Umgekehrt gilt für eine bestehende Instanz: den HSM-Job **nicht** löschen und **nicht** erneut ausführen, wenn das Token-Volume erhalten bleiben soll. Sonst besteht das Risiko einer neuen Instanz-DID, und alle darauf gepinnten Vertrauensanker sind ungültig.

- **Bestehende Jobs sind unveränderlich.** `apply` scheitert an ihnen; sie sind vorher zu löschen.
- Ein `pullPolicy: Always` zieht nur bei Pod-Erzeugung neu. Nach einem Apply, der die Manifeste nicht verändert hat, ist ein Rollout-Restart nötig:

```
kubectl rollout restart deploy -n <ns> <release>-digital-contracting-service \
  <release>-digital-contracting-service-pdf-core <release>-orice
```

A14 Deinstallation

A14.1 Vorher lesen

`helm uninstall` löscht **alle** PersistentVolumeClaims dieses Release. Keiner der PVCs im Chart trägt eine `helm.sh/resource-policy: keep`-Annotation. Damit verschwinden:

- der SoftHSM2-Token und mit ihm **die Identität der Instanz**. Ein neu aufgebautes Deployment hat eine neue DID; jedes Vertrauensdokument und jeder Peer, der die alte DID kennt, ist danach falsch;
- das IPFS-Repository und damit sämtliche Artefakt-Chiffre **einschließlich des Audit-Trail-Inhalts**;
- die Postgres-Volumes und damit die gewrappten CEKs, ohne die auch ein gesichertes IPFS-Repository nicht mehr lesbar ist;
- das ORCE-Volume und damit die **Archiv-Notar-Belegkette** (`/data/archive-audit-events.jsonl` , wogegen die Archiv-Integritätsprüfung vergleicht) sowie den Seriennummernzähler der lokalen TSA. Auf diesem Volume liegen Nachweise, die sich aus dem Deployment nicht wiederherstellen lassen.

Vor jeder Deinstallation, die nicht ein Wegwerf-Cluster betrifft, ist die Sicherung nach `docs/backup-integration-guide.md` durchzuführen und zu verifizieren.

Nicht mit gelöscht wird die Trust-ConfigMap, wenn sie über `oid4vp.trust.existingConfigMap` referenziert ist. Helm verwaltet sie nicht.

A14.2 Prozedur

Voraussetzungen: Sicherung liegt vor und ist geprüft; Peers sind über den Wegfall informiert, wenn die Instanz föderiert war.

Schritte:

1. Release entfernen:

```
helm uninstall <release> --namespace <ns>
```

2. Verbliebene Objekte prüfen und gezielt entfernen. Jobs, die einen PVC noch halten, blockieren dessen Löschung:

```
kubectl -n <ns> get pods,jobs,pvc,ingress -l app.kubernetes.io/instance=<release>
```

3. Issuer-Releases desselben Namespace, sofern sie mit entfallen sollen:

```
helm uninstall dcs-issuer --namespace <ns>
```

4. Nicht von Helm verwaltete Objekte: die Trust-ConfigMap, vorab angelegte Secrets und von Hand erzeugte Ingress-Regeln.

Verifikation:

```
helm -n <ns> list
kubectl -n <ns> get all,pvc
```

Die erste Ausgabe darf das Release nicht mehr enthalten, die zweite keine Objekte mit dem Release-Label.

Fehlerbilder:

Symptom	Ursache	Behebung
Ein PVC bleibt in <code>Terminating</code>	<code>pvc-protection</code> : ein Pod referenziert ihn noch, auch ein <code>Succeeded</code> -Job-Pod	Job-Pods löschen, dann das Volume
Nach der Deinstallation antworten Ingress-Regeln weiter	Von Hand angelegte oder von einem anderen Release stammende Regeln	Über <code>kubectl -n <ns> get ingress</code> suchen und einzeln entfernen

A14.3 `uninstall.sh` ist nicht der Betreiberweg

`deployment/helm/uninstall.sh` gehört zum Node-RED-Deployment-Paket unter `deployment/node-red/` , das DCS-Instanzen aus einem Flow heraus anlegt und wieder abräumt. Es wird von dort mit `<kubeconfig> <instanzname>` aufgerufen und leitet daraus fest Namespace `digital-contracting-service-<instanzname>` und Release `digital-contracting-service` ab. Danach **löscht es den gesamten Namespace**.

Gegen ein Deployment, das nach A8 installiert wurde, passt dieses Namensschema nicht, und der Namespace-Löschbefehl trifft auch Objekte, die nicht zum Release gehören. Für eine Instanz aus diesem Leitfaden gilt A14.2.

Teil B: Lokales Entwickler- und Testsetup

Dieser Teil beschreibt Werkzeuge, die in Produktion nichts zu suchen haben: lokal laufende Prozesse ohne Container, im Repository liegende Schlüssel-Fixtures und Testcredentials. Sie sind hier zulässig, weil ein Entwicklungsstack keine Vertraulichkeit trägt. Keine der Werte-Dateien aus diesem Teil darf in einem erreichbaren Cluster verwendet werden.

B1 Voraussetzungen

- Rancher Desktop (oder eine gleichwertige lokale Kubernetes-Umgebung) mit NodePort-Weiterleitung auf `localhost`
- Go 1.25+ mit `air` (`backend/go.mod` fordert `go 1.25.7`)
- Node.js 22+ (Bau-Image und CI verwenden 22)
- Python 3.10+, Goa v3, `make`, `curl`, OpenSSL
- SoftHSM2 mit `libsofthsm2.so` und `softhsm2-util`, dazu OpenSC für `pkcs11-tool`. Die Entwicklungsschlüssel liegen in einem PKCS#11-Token, nicht in Dateien.
- `helm`, `kubect1`, `docker`
- Für die BDD-Suite zusätzlich `kind` (die CI verwendet v0.23.0), `git` (der Runner klonet den `bdd-executor`, falls er nicht neben dem Repository liegt) sowie `nc` und `fuser` (der Runner prüft und räumt damit seine lokalen Ports)

B2 Vollständiger Stack mit einem Befehl

Voraussetzung: Ein SoftHSM-Token muss vorhanden sein. Das Backend öffnet beim Start ein PKCS#11-Token und kommt ohne eines nicht über den Bootstrap hinaus. Es gibt keinen Software-Fallback, auch lokal nicht. `dev-stack.sh` legt das Token in Schritt 6 selbst an (`$HOME/.dcs/softhsm-8991`, Label `dcs`) und ist dabei idempotent: Ein vorhandenes Token und vorhandene Schlüssel bleiben unangetastet. Dafür müssen `softhsm2-util` und `pkcs11-tool` installiert sein (B1); fehlen sie, versucht das Provisionierungsskript, sie per `apt-get` nachzuinstallieren, und braucht dafür `sudo`.

```
bash dev-stack.sh
```

Das Skript, in dieser Reihenfolge:

1. `helm dependency build --skip-refresh deployment/helm`, dann `helm upgrade --install dcs deployment/helm -f deployment/helm/values.dev.yml`
2. wartet auf den Federated Catalogue, falls die Werte-Datei ihn aktiviert. Das Katalog-Readiness-Gate des Backends macht genau einen Versuch; ein zu früh gestarteter Backend-Prozess stirbt daran
3. exportiert das TSA-Zertifikat aus dem Secret `dcs-orce-tsa-material` nach `backend/certs/dev/orce-tsa-cert.pem`
4. wartet auf den Status-List-Dienst, installiert das `testWallet-venv` (`make -C testWallet install`) und ruft dessen Listen-Initialisierung auf.
Dieser Aufruf ist derzeit tot: `make -C testWallet ensure-statuslist` startet ein Skript (`testWallet/scripts/ensure_statuslist_for_dev.py`), das nicht mehr existiert. Der Aufruf schlägt fehl, und `dev-stack.sh` bricht wegen `set -e` an dieser Stelle ab. Bis das im Skript behoben ist, kommt der Stack-Start über diesen Schritt nicht hinaus
5. kopiert `backend/.env.dev1` nach `backend/.env`
6. provisioniert das lokale SoftHSM2-Token unter `$HOME/.dcs/softhsm-8991` mit den fünf Instanzschlüsseln und erzeugt `backend/certs/dev/did-8991.json` mit `gendid`
7. stellt die C2PA-x5chain und eine leere CRL für pdf-core aus
8. startet pdf-core mit `air`, wartet auf `http://localhost:8080/version`
9. startet das EU-DSS-Demo-Image auf `http://localhost:18099` (abschaltbar mit `DCS_DEV_DSS=0`)
10. startet den Vite-Dev-Server, generiert die Goa-Schicht (`goa gen`) und startet das Backend mit `air`

Backend: `http://localhost:8991`. Vite: `http://localhost:5173`, proxied `/api` auf das Backend. Beenden mit `Strg+C` im selben Terminal.

`values.dev.yml` setzt `replicaCount: 0` und `pdfCore.enabled: false`. Backend und pdf-core laufen nativ statt im Cluster, deshalb wird auch kein In-Cluster-Token provisioniert; das Token liegt auf dem Entwicklungsrechner.

Die exponierten NodePorts stehen im Kopf von `values.dev.yml` (Instanz A) und `values.dev2.yml` (Instanz B); die beiden Instanzen belegen disjunkte Portbereiche.

Verifikation: `curl http://localhost:8991/api/...` antwortet und `/tmp/backend-live.log` enthält `HTTP server listening`. Dass das Token steht, zeigt zusätzlich:

```
SOFTHSM2_CONF=$HOME/.dcs/softhsm-8991/softhsm2.conf softhsm2-util --show-slots | grep -A2 'Label:'
```

Ein Slot mit dem Label `dcs` muss erscheinen. Beendet sich einer der drei Prozesse, beendet sich `dev-stack.sh` mit dessen Status; bei Backend und pdf-core gibt es vorher die letzten 200 Logzeilen des betroffenen Prozesses aus.

Wer den Stack von Hand statt über `dev-stack.sh` startet, muss die Reihenfolge einhalten. Auf ein fehlendes Token wartet das Backend nur (es pollt, siehe A12.1); die übrigen drei Abhängigkeiten sind harte Startgatter und führen sonst zu einem Prozess, der sofort wieder stirbt:

1. Das SoftHSM-Token muss existieren, bevor das Backend startet. Von Hand: `bash scripts/hsm-provision.sh $HOME/.dcs/softhsm-8991 dcs 1234 12345678 .`
2. Der Federated Catalogue muss antworten, bevor das Backend startet. Das Readiness-Gate macht genau **einen** Versuch. Das ist Absicht, damit ein kaputtter Katalog nicht durch Wiederholungen verdeckt wird.
3. **pdf-core muss laufen** (`http://localhost:8080`). Das Backend probiert `/version` bis zu drei Minuten lang und beendet sich danach. Ein manueller Ablauf, der nur Backend und Frontend startet, funktioniert nicht.
4. Der Status-List-Dienst muss erreichbar sein, ebenfalls mit einem Drei-Minuten-Fenster, danach fatal.

Punkt 3 wird am häufigsten übersehen. pdf-core ist ein eigener Prozess (`cd pdf-core && air`) und kein Teil des Backends.

Zweite Instanz. `bash dev-stack2.sh` startet Instanz B auf `:8992` mit Frontend auf `:5174` und eigenem Token unter `~/dcs/softhsm-8992`. Sie **teilt sich pdf-core mit Instanz A** und prüft dessen Erreichbarkeit beim Start. Instanz A muss also zuerst laufen.

B3 BDD- und E2E-Suiten

Alle Ziele liegen in `tests/bdd/Makefile`.

```
# Cluster aufbauen, beide Instanzen deployen, Umgebung vorbereiten
make -C tests/bdd kind_up

# Suite gegen den stehenden Cluster ausführen (beliebig oft)
make -C tests/bdd run_bdd_kind_once
make -C tests/bdd run_bdd_kind_once F=features/<pfad>

# Cluster abbauen und Artefakte aufräumen
make -C tests/bdd kind_down
```

Einmaliger Vollzyklus, wie ihn die CI fährt:

```
make -C tests/bdd run_bdd_kind_ci
```

Weitere Ziele: `run_e2e_kind_ci` (Playwright statt behave), `run_bdd_audit_kind_ci` und `kind_up_audit` / `run_bdd_audit_kind_once` für den isolierten Audit-Gate mit nur einer Instanz, `kind_delete` zum reinen Löschen des Clusters. `make help` listet die gebräuchlichen.

Zwei Ziele nicht verwenden: `run_bdd_kind_all` ruft ein `run_bdd_helm_all` auf, das der Makefile nicht definiert; der Aufruf bricht mit `No rule to make target` ab. Und ein `run_bdd_helm_dev` gibt es nicht. Gegen ein bereits laufendes Release ist `run_bdd_helm` das richtige Ziel (B4.2).

`run_bdd_helm` prüft vor dem Testlauf, dass der Stack wirklich antwortet, und bricht sonst mit einer Diagnose ab. Die Wartebedingungen sind erprobte Startsignale; wer den Stack von Hand prüft, kann sich an derselben Reihenfolge orientieren:

1. Deployment und Backend-Pod von Instanz A sind verfügbar bzw. `ready` (je 180 s Frist); scheitert das, folgt eine Federated-Catalogue-Diagnose.
2. Der Login-Endpunkt antwortet **durch den Trafik-Ingress** auf Port 18080. Eine Go-Standard-404-Antwort wird als eigenes Fehlerbild erkannt: Der Host-Port erreicht den Ingress nicht.
3. Instanz B, falls vorhanden: Deployment verfügbar (300 s) und HTTP über `dcs-b.localhost` (bis 120 s). Fehlt sie, warnt der Runner ausdrücklich, statt still zu übergehen; die `@two-instance`-Szenarien schlagen dann fehl.
4. Das DSS-Deployment ist verfügbar (Frist 420 s; das Tomcat-Bundle bootet langsam, seine Readiness-Probe beginnt erst nach 90 s).
5. ORCE ist verfügbar, das Archiv-Audit-Token ist aus dem laufenden Pod lesbar, der Audit-Log-Endpunkt antwortet, und der Contract-Target-Flow antwortet durch den Ingress nicht mit 404.

Danach richtet der Runner Port-Weiterleitungen für **PostgreSQL (5432), den DCS-Service (18991), DSS (18099) und ORCE (18880)** ein. Die DCS-Weiterleitung dient den Endpunkten, die der Ingress nicht routet, etwa `/metrics`. Keycloak wird nicht weitergeleitet. Vorher beendet der Runner jeden Prozess, der die Ports 5432, 18991 oder 18880 belegt; ein lokal laufender PostgreSQL auf 5432 wird dabei mit beendet. Der Standardlauf schließt `@isolated_stack`-Szenarien aus, sie laufen nur in ihren eigenen Zielen. Ein Lauf, der null Szenarien ausgeführt hat, gilt als Fehlschlag, nicht als grün.

Der Ablauf von `kind_deploy` in Kurzform: kind-Cluster anlegen oder wiederverwenden, die vorab gebauten DCS- und pdf-core-Images sowie das DSS-Image hineinladen, Helm-Repositories aktualisieren und Abhängigkeiten bauen, Prometheus-Stack-CRDs anwenden, Namespace `dcs-bdd` anlegen, ORCE-Volume vorbereiten, Host-Adresse für den Checkpoint-Sink bestimmen, `helm upgrade --install dcs` mit `values.bdd.yml` (öffentliche Origin-, Hydra- und Checkpoint-Sink-Adressen kommen per `--set` dazu), CoreDNS-Rewrite für `dcs-a.localhost` / `dcs-b.localhost`,

Rollout-Restart der frisch gebauten Workloads, warten auf das x5chain-Secret (bis zu 20 Minuten), warten auf den Rollout. `kind_deploy_b` legt anschließend Instanz B mit `values.bdd2.yml` in denselben Namespace, strikt danach, weil B mehrere von A bereitgestellte Dienste nutzt (Traefik, ORCE samt TSA und Trust-PDP, Federated Catalogue, Status-Listen-Dienst, DSS) und einige davon beim eigenen Start aktiv anfragt.

Den HTTPS-Sink für die Checkpoint-Szenarien betreibt die Suite selbst: ein Listener im behave-Prozess auf Host-Port 18443 (`BDD_CHECKPOINT_SINK_PORT`). `kind_deploy` setzt `orce.checkpointSink.url` auf eine vom Cluster aus erreichbare Adresse dieses Hosts (`host.docker.internal`, unter WSL die Rancher-Desktop-veth-Adresse) und bricht ab, wenn sich keine bestimmen lässt. Port 18443 muss auf dem Host frei sein.

Das Deployment läuft bewusst **ohne** `--wait`: Helm hält `post-install`-Hooks zurück, bis alle Deployments bereit sind, und das Backend wird erst bereit, wenn der HSM-Hook sein Secret erzeugt hat. Deshalb wartet der Makefile getrennt, zuerst auf das Secret und dann auf den Rollout.

B4 Abweichungen der lokalen Cluster-Variante

`values.bdd.yml` ist auf kind zugeschnitten. Gegen einen anderen lokalen Cluster gelten zwei Abweichungen.

B4.1 Rancher-Desktop-k3s statt kind: Traefik-hostPort

Symptom. Der Traefik-Pod bleibt `Pending`. In der Folge wird der Statuslist-Sidecar **beider** DCS-Pods nie `ready`, weil dessen Readiness-Probe eine echte HTTP-Route durch den Ingress verlangt. Der Backend-Pod bleibt dadurch dauerhaft nicht bereit, obwohl der Backend-Container selbst in Ordnung ist.

Ursache. `values.bdd.yml` setzt `traefik.ports.web.hostPort: 18080` für kind. Auf k3s belegt Klippers `svclb` diesen Port bereits, der Pod findet keinen Node und bleibt ungeplant.

Behebung. Den hostPort beim Deployment neutralisieren:

```
helm upgrade --install dcs deployment/helm -n dcs-bdd \
  -f deployment/helm/values.bdd.yml \
  --set traefik.ports.web.hostPort=null
```

Verifikation.

```
kubectl -n kube-system get pod -l app.kubernetes.io/name=traefik
kubectl -n dcs-bdd get pods -o 'custom-
columns=POD:.metadata.name,CONTAINER:.status.containerStatuses[*].name,READY:.status.containerStatuses[*].ready'
```

Der Traefik-Pod muss `Running` sein, und im DCS-Pod müssen **beide** Container `true` melden.

B4.2 KUBECONFIG-Bindung der `kind_*`-Ziele

Jedes `kind_*`-Ziel im Makefile exportiert `KUBECONFIG` auf `tests/bdd/.tmp/kind-kubeconfig`. Das ist Absicht: sonst landen Deployments dort, wohin der aktuelle Kontext des Entwicklers zeigt, während der Datenpfad der Suite (Port 18080) immer den kind-Cluster erreicht.

Die Folge: **Gegen einen anderen Cluster als kind darf keines dieser Ziele verwendet werden.** Dort ist `make run_bdd_helm` direkt aufzurufen. Es gehört nicht zur `kind_*`-Gruppe und respektiert die vorhandene `KUBECONFIG`:

```
make -C tests/bdd run_bdd_helm K8S_NAMESPACE=<ns> HELM_RELEASE=<release>
```

Weitere überschreibbare Variablen mit Vorgaben: `K8S_NAMESPACE` (`default`), `HELM_RELEASE` (`dcs`), `FEATURES_PATH` (`features`), `BDD_INGRESS_PORT` (`18080`), `LOCAL_FORWARD_PORT` (`18991`), `RUN_MODE` (`bdd`, alternativ `e2e`), `HELM_TIMEOUT` (`20m`).

Unter WSL legt der Makefile das Python-venv per Vorgabe unter `$HOME/.dcs-bdd-venv` an statt im Repository. Ein venv auf dem 9P-Dateisystem verlangsamt jeden Python-Import erheblich. `VENV_PATH=<pfad>` überschreibt das.

B5 Was aus Teil B nicht in Produktion gehört

Mechanismus	Warum er lokal existiert	Warum er in Produktion unzulässig ist
<code>DCS_ALLOW_DEV_TRUST=true</code>	Erlaubt das im Image liegende Trust-Fixture	Dessen Issuer-Schlüssel liegen im Repository; jeder mit einer Kopie kann akzeptierte Credentials prägen
<code>JWT_ALG_NONE_SUPPORTED</code> , <code>AUTH_INSECURE_COOKIES</code>	Vereinfachen den Testaufbau	Beide entfernen Sicherheitseigenschaften
<code>image.pullPolicy: Never</code>	Nutzt lokal in kind geladene Images	Kein Pod startet in einem Cluster ohne diese Images
<code>hydra.config.dev: true</code>	Erlaubt <code>http</code> -Issuer-URLs	Erlaubt unverschlüsselte OIDC-Endpunkte
Fest verdrahtete Testcredentials in den <code>values.dev*</code> / <code>values.bdd*</code> -Dateien (Hydra-Clients, Katalog-Secret, System-Token, Archiv-Token)	Reproduzierbare Testläufe	Öffentlich bekannte Werte
<code>statusListLocalhostProxy</code>	Löst eine <code>localhost</code> -Auflösung im Testcluster	Ein Sidecar ohne Zweck außerhalb dieses Aufbaus
<code>serviceMonitor.enabled</code>	Bindet die Testinstanz an den Prometheus des kind-Clusters	Das gerenderte Objekt trägt fest den Namespace <code>dcx-bdd</code> und den Selektor <code>release: prometheus</code> . In jedem anderen Namespace ist es wirkungslos oder wird abgelehnt. Das Scrape-Ziel für den Betrieb legt der Betreiber selbst an (A12.1)
SoftHSM2 im Cluster	Kein HSM nötig	Ein Softwaretoken; die privaten Schlüssel liegen auf einem Cluster-Volume
<code>orce.syntheticPeer.enabled</code>	Synthetische Peer-Identitäten für Föderationsszenarien	Testkulisser; belegt zusätzlich Services gegen eine Namespace-Quota
<code>postgresql.persistence.enabled: false</code> (Chart-Vorgabe)	Schnelle Wegwerf-Cluster	Ein Pod-Neustart verliert jeden Vertrag